

LeaseGPT — Complete Tutorial

A comprehensive, beginner-friendly guide to building a production-grade lease document Q&A system.

LeaseGPT Tutorial - Part 1: Foundations, Setup, and Database

Welcome to Part 1 of the LeaseGPT tutorial series! In this tutorial, we will build a full-stack, AI-powered web application from scratch. We're going to build **LeaseGPT** — a system that allows users to upload PDF or image copies of their leases, and then ask natural language questions about them through a ChatGPT-like interface.

This tutorial is designed specifically for **complete beginners** to AI engineering. We will explain every concept, every tool, and every line of code. If you have some basic programming experience but have never touched AI, Docker, or Cloud architectures before, you are in the right place!

Chapter 0: What Are We Building? (The Big Picture)

Before we write any code, let's understand the core concepts and technologies we'll be using.

What is a Large Language Model (LLM)?

Imagine a person who has read millions of books, articles, and websites, and has learned the patterns of human language so well that they can predict, word by word, what should logically come next in a sentence. That's essentially what a Large Language Model is. It's a massive mathematical equation (a neural network) that takes in text (your prompt) and predicts the most likely response based on the massive amounts of data it was trained on.

It does not "think" or "know" facts like a human does; it generates text based on patterns. When you ask it a question, it uses probabilities to guess the right words to output.

What is RAG (Retrieval-Augmented Generation)?

Why can't we just give a lease document to the LLM and ask it questions directly?

1. **Memory Limits (Context Window):** LLMs have a limit on how much text they can process at once. A massive commercial lease might exceed this limit.
2. **Cost:** Even if it fits, sending massive documents to an LLM every time you ask a question costs a lot of money and computational power.
3. **Accuracy (Hallucinations):** LLMs sometimes make things up (hallucinate).

RAG is the solution.

Think of an LLM as a very smart person locked in a room without internet access. If you ask them a highly specific question about your personal lease, they won't know the answer and might guess (hallucinate).

Now, what if you give that person a librarian (the **Retriever**)? When you ask a question, the librarian quickly searches through your lease, finds the 3 most relevant pages, and hands them to the smart person. The smart person then reads those 3 pages and gives you an accurate answer.

That's RAG! We **retrieve** relevant information from our database and **augment** the LLM's prompt with that information so it can **generate** an accurate response.

What is Hybrid RAG?

There are two main ways to search text:

1. **Keyword Search:** Looking for exact words (like finding the word "pet" in a document).
2. **Semantic Search:** Looking for meaning. If you search for "dogs allowed?", semantic search is smart enough to find the paragraph talking about "canines permitted on the premises," even if the exact words "dogs" or "allowed" aren't there.

Hybrid RAG combines both methods to give us the best of both worlds—pinpoint accuracy for exact terms, and contextual understanding for concepts.

What is a Vector Embedding?

How does an AI understand "meaning"? By turning words into numbers.

A vector embedding is a list of numbers (a vector) that represents the meaning of a piece of text.

Imagine a 2D graph where the X-axis is "How much is this related to animals?" and the Y-axis is "How much is this related to homes?".

The word "Dog" might be at `[0.9, 0.1]`. The word "Apartment" might be at `[0.1, 0.9]`. The phrase "Pet Rent" might be right in the middle at `[0.8, 0.8]`.

Because "Dog" and "Pet Rent" are close to each other on the graph, the computer knows their meanings are related.

In this project, we use embeddings with **768 dimensions** (not just 2!). This allows the AI to capture incredibly complex nuances in meaning.

What is an AI Agent? What is LangGraph?

An **AI Agent** is an LLM that has been given tools and a loop to think. Instead of just answering a question immediately, an agent can say: "To answer this, I first need to use my Search Tool. Let me do that... Okay, I got the results. Now let me analyze them... Okay, here is the final answer."

LangGraph is a Python library that helps us build these agents. It lets us define the agent's decision-making process as a flowchart (a graph) with nodes (steps) and edges (rules for moving to the next step).

What is Ollama?

Usually, to use a powerful LLM, you have to pay companies like OpenAI (ChatGPT) or Anthropic (Claude) for every message.

Ollama is a free, open-source tool that lets you run powerful LLMs directly on your own computer. It means everything stays private (your lease data never leaves your laptop) and it's completely free!

High-Level System Architecture

Here is how all the pieces fit together:

```

graph TD
    %% User Flow
    User((User)) -->|Asks Question| Frontend[Frontend UI  
Next.js / React]
    Frontend -->|API Request| Backend[Backend API  
FastAPI / Python]

    %% AI Agent Flow
    Backend --> Agent[AI Agent  
LangGraph]
    Agent <-->|Uses Tools| Retriever[Hybrid Retriever]
    Agent <-->|Generates Answer| LLM[Local LLM  
Ollama]

    %% Database Flow
    Retriever <-->|Semantic & Keyword Search| DB[(Database  
PostgreSQL + pgvector)]
    Backend <-->|Chat History| Redis[(Cache  
Redis)]

    %% Data Ingestion Flow
    Upload[PDF Upload] --> S3[AWS S3  
Storage]
    S3 --> Lambda[AWS Lambda  
Text Extraction]
    Lambda -->|Extracted Text| Chunker[Text Chunker]
    Chunker -->|Creates Embeddings| Embed[Embedding Model  
Ollama]
    Embed -->|Saves Vectors| DB

    classDef default fill:#f9f9f9,stroke:#333,stroke-width:2px;
    classDef primary fill:#e1f5fe,stroke:#03a9f4,stroke-width:2px;
    classDef ai fill:#f3e5f5,stroke:#9c27b0,stroke-width:2px;
    classDef db fill:#e8f5e9,stroke:#4caf50,stroke-width:2px;

    class Frontend,Backend primary;
    class Agent,LLM,Embed ai;
    class DB,Redis db;

```

The Tech Stack Explained

- **Next.js + TypeScript + TailwindCSS:** The frontend framework. Next.js helps us build fast React websites. TypeScript ensures our code doesn't have silly typos. TailwindCSS makes styling easy.
- **Python + FastAPI + WebSockets:** The backend. Python is the king of AI languages. FastAPI is a super-fast web framework for Python. WebSockets allow for real-time streaming of chat responses.
- **LangGraph & LlamaIndex:** AI libraries. LangGraph controls the agent's logic, and LlamaIndex helps us handle documents and RAG.
- **PostgreSQL + pgvector:** Our database. PostgreSQL stores our users and chat history. The `pgvector` extension allows it to store and search our 768-dimension vector embeddings.

- **Redis:** A super-fast temporary database (cache) to store active chat sessions.
- **AWS (S3, Lambda, Step Functions, Textract):** Cloud services we'll use later to process uploaded PDFs.
- **Docker & Kubernetes:** Tools to package our application so it runs identically on any computer.
- **Ollama:** Runs our local LLM (Llama 3) and our embedding model (nomic-embed-text) for free.

Chapter 1: Prerequisites & Environment Setup

Since this tutorial is for Windows, we will set everything up perfectly for a Windows environment.

IMPORTANT

Take your time with this chapter. 90% of beginner programming headaches come from a poorly configured environment!

1. Python 3.12

Python is the language we'll use for the backend.

1. Go to python.org/downloads
2. Download Python 3.12 (do not use 3.13 yet, as some AI libraries are slow to update).
3. **CRITICAL:** When running the installer, check the box that says "**Add Python to PATH**" at the bottom of the first screen.
4. Verify by opening Command Prompt (CMD) or PowerShell and typing: `python --version`

2. Node.js 20 LTS

Node.js lets us run JavaScript outside the browser, which is needed for our Next.js frontend.

1. Go to nodejs.org
2. Download the **20.x LTS** (Long Term Support) version.
3. Install with default settings.
4. Verify in terminal: `node --version`

3. Docker Desktop

Docker lets us run applications in isolated "containers". Think of a container as a tiny, lightweight, disposable virtual machine that has exactly what our app needs to run.

1. Go to docker.com/products/docker-desktop
2. Download and install Docker Desktop for Windows.

3. Ensure WSL 2 (Windows Subsystem for Linux) is enabled when prompted during installation. This makes Docker run incredibly fast on Windows.
4. Restart your computer if prompted.
5. Open Docker Desktop to make sure it's running.
6. Verify in terminal: `docker --version`

4. Git

Git tracks changes to our code (version control).

1. Go to git-scm.com/download/win
2. Download and install with default settings.
3. Verify in terminal: `git --version`

5. VS Code

The best code editor for this stack.

1. Go to code.visualstudio.com
2. Download and install.
3. Open VS Code, click the "Extensions" icon on the left (or press Ctrl+Shift+X), and install these extensions:
 - **Python** (by Microsoft)
 - **ESLint** (by Microsoft)
 - **Tailwind CSS IntelliSense** (by Tailwind Labs)
 - **Docker** (by Microsoft)
 - **Prisma** (by Prisma - useful for database schemas)

6. Ollama

This is the magic tool that will run our AI models locally.

1. Go to ollama.com/download
2. Download for Windows and install.
3. Open a terminal and run `ollama --version` to verify.
4. Now, we need to download our models. In your terminal, run:

```
bash ollama pull llama3.1:8b
```

This downloads the Llama 3.1 model (the brain that will answer questions). It's about 4.7GB, so it might take a while depending on your internet.

5. Next, download the embedding model:

```
bash ollama pull nomic-embed-text
```

This downloads a smaller model specifically designed to turn text into numbers (vector embeddings). It's about 274MB.

7. AWS CLI

We will use Amazon Web Services later in the tutorial.

1. Download the AWS CLI MSI installer for Windows from aws.amazon.com/cli.
2. Run the installer.
3. We will configure this in a later chapter when we actually need it.

8. Kubernetes (kubectl and kind)

We will use these later for deploying our app.

For now, let's just install them.

1. Open PowerShell **as Administrator**.
2. If you don't have a package manager like `winget` or `choco`, the easiest way on Windows is via Docker Desktop.
 - Open Docker Desktop Settings (gear icon) -> Kubernetes -> Check "Enable Kubernetes" -> Apply & Restart.
 - This installs `kubectl` automatically!

Chapter 2: Project Scaffolding

Now let's build the skeleton of our project. We are using a **monorepo** structure, meaning our frontend, backend, and infrastructure code will all live in one single Git repository.

Open your terminal and let's create the folders:

```
mkdir leasegpt
cd leasegpt
mkdir frontend backend infra docs
```

Here is what these folders are for:

- `frontend/` : Our Next.js React user interface.
- `backend/` : Our Python FastAPI server and AI Agent code.
- `infra/` : Docker and Kubernetes configuration files.
- `docs/` : Documentation (like this tutorial!).

Now, open this `leasegpt` folder in VS Code.

Root Files

Let's create some files in the main `leasegpt` folder.

1. `.gitignore`

This file tells Git which files it should NOT upload to the internet (like secret passwords or massive auto-generated folders).

Create a file named `.gitignore` in the root folder:

```

# .gitignore

# Python
__pycache__/*
*.py[cod]
*$py.class
*.so
.Python
env/
build/
develop-eggs/
dist/
downloads/
eggs/
.eggs/
lib/
lib64/
parts/
sdist/
var/
wheels/
share/python-wheels/
*.egg-info/
.installed.cfg
*.egg
MANIFEST
.env
.venv
venv/
venv.bak/

# Node
node_modules/
npm-debug.log
yarn-error.log
yarn-debug.log
.pnpm-debug.log

# Next.js
.next/
out/

# Docker
.docker/

# OS Files
.DS_Store
Thumbs.db

# IDEs
.vscode/
.idea/

# Environment Variables - NEVER COMMIT SECRETS!
.env
.env.local
.env.*.local

```

2. Makefile

A Makefile is a script that gives us short commands to run long, annoying terminal commands.

Note: On Windows, you might need to install `make` (e.g., via `choco install make`), or you can just read the commands from this file and run them manually.

Create a file named `Makefile` in the root folder:

```
# Makefile

.PHONY: dev-up dev-down logs backend-shell db-shell

# Start all Docker services in the background (-d means detached)
dev-up:
    docker-compose -f infra/docker/docker-compose.yml up -d

# Stop all Docker services
dev-down:
    docker-compose -f infra/docker/docker-compose.yml down

# View logs from all services
logs:
    docker-compose -f infra/docker/docker-compose.yml logs -f

# Open a terminal inside the running backend container
backend-shell:
    docker-compose -f infra/docker/docker-compose.yml exec backend bash

# Open a database connection in the terminal
db-shell:
    docker-compose -f infra/docker/docker-compose.yml exec postgres psql -U postgres -c
```

3. .env.example

We use `.env` files to store secret passwords and configuration on our local computer. We NEVER upload `.env` to Git. Instead, we upload `.env.example` so other developers know what variables they need to set.

Create `.env.example`:

```
# .env.example

# --- Database Settings ---
# URL format: postgresql+asyncpg://username:password@host:port/database_name
DATABASE_URL=postgresql+asyncpg://postgres:postgres@localhost:5432/leasegpt
POSTGRES_USER=postgres
POSTGRES_PASSWORD=postgres
POSTGRES_DB=leasegpt

# --- Redis Settings ---
REDIS_URL=redis://localhost:6379/0

# --- AI Models ---
# The URL where Ollama is running locally
OLLAMA_BASE_URL=http://localhost:11434
# The LLM we pulled earlier
LLM_MODEL=llama3.1:8b
# The embedding model we pulled earlier
EMBEDDING_MODEL=nomic-embed-text

# --- Backend API ---
# Secret key for JWT tokens (change in production!)
API_SECRET_KEY=super_secret_dev_key_123
ENVIRONMENT=development
```

✓ TIP

Copy `.env.example` and rename the copy to `.env`. This `.env` file is where your app will actually read its configuration from!

4. README.md

Create a basic `README.md` :

```
# LeaseGPT

An AI-powered application that allows users to ask questions about their lease document

## Tech Stack
- Frontend: Next.js, TailwindCSS
- Backend: FastAPI, LangGraph, LlamaIndex
- Database: PostgreSQL (with pgvector), Redis
- AI: Ollama (Local Llama 3 & nomic-embed-text)
```

Now, let's initialize Git:

Open your terminal in VS code (Ctrl+`) and type:

```
git init
git add .
git commit -m "Initial commit: Project scaffolding"
```

Chapter 3: Database Layer (PostgreSQL + pgvector)

Our app needs a place to store data. We need to store structured data (like Users and Chat Sessions) and we need to store our vector embeddings.

Setting up Docker Compose

Instead of installing PostgreSQL and Redis manually on Windows (which is messy), we will use Docker to run them in containers.

Create the folder path `infra/docker/` and then create `docker-compose.yml` inside it:

```
# infra/docker/docker-compose.yml

version: '3.8'

# 'services' defines the different containers we want to run
services:

  # 1. Our PostgreSQL Database
  postgres:
    # We use a specific image that already has the pgvector extension installed!
    image: pgvector/pgvector:pg16
    container_name: leasegpt-postgres
    # These map to our .env variables
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: leasegpt
    # We map port 5432 inside the container to 5432 on our Windows machine
    ports:
      - "5432:5432"
    # Volumes save our data. If we don't do this, the database wipes clean every time w
    volumes:
      - postgres_data:/var/lib/postgresql/data
    # A healthcheck ensures the database is fully booted before other services try to c
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d leasegpt"]
      interval: 5s
      timeout: 5s
      retries: 5

  # 2. Redis (Our fast in-memory cache)
  redis:
    image: redis:7-alpine
    container_name: leasegpt-redis
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 5s
      retries: 5

# We must declare the volumes we used above
volumes:
  postgres_data:
  redis_data:
```

What just happened? We wrote a recipe that tells Docker: "Hey, go download a PostgreSQL database (specifically one with the vector extension) and a Redis cache, set them up with these passwords, open these ports so my computer can talk to them, and make sure their data survives a restart."

Start it up by running this in your terminal:

```
docker-compose -f infra/docker/docker-compose.yml up -d
```

(The `-d` means "detached" so it runs in the background and frees up your terminal).

Python Backend Setup

Now let's set up the Python side of our database.

In your terminal, navigate to the backend folder:

```
cd backend
```

Let's create our python environment and configuration file. Create `pyproject.toml` in the `backend/` folder:

```
# backend/pyproject.toml

[tool.poetry]
name = "leasegpt-backend"
version = "0.1.0"
description = "FastAPI backend for LeaseGPT with LangGraph and PostgreSQL"
authors = ["Your Name <you@example.com>"]

[tool.poetry.dependencies]
python = "^3.12"
# FastAPI is our web framework
fastapi = "^0.110.0"
# Uvicorn is the server that runs FastAPI
uvicorn = {extras = ["standard"], version = "^0.29.0"}
# Pydantic handles our settings and data validation
pydantic = "^2.6.0"
pydantic-settings = "^2.2.0"
# SQLAlchemy is our ORM (Object Relational Mapper) - talks to the DB
sqlalchemy = "^2.0.29"
# asyncpg is the driver that lets SQLAlchemy talk to PostgreSQL asynchronously
asyncpg = "^0.29.0"
# Alembic handles database migrations (schema changes)
alembic = "^1.13.1"
# pgvector is the python package to handle our vector embeddings in Postgres
pgvector = "^0.2.5"
# Redis client for python
redis = {extras = ["hiredis"], version = "^5.0.3"}

[build-system]
requires = ["poetry-core"]
build-backend = "poetry.core.masonry.api"
```

To manage our Python packages easily, we'll use a standard virtual environment and pip for this tutorial (though you can use Poetry if you prefer).

In the `backend` folder, run:

```
# Create a virtual environment
python -m venv venv

# Activate it (Windows Command)
.\venv\Scripts\activate

# Upgrade pip
python -m pip install --upgrade pip

# Install dependencies from the toml file (pip can read dependencies from poetry files)
pip install fastapi "uvicorn[standard]" pydantic pydantic-settings sqlalchemy asyncpg a
```

Application Configuration

Create the folders `app/db` and `app/models` inside `backend/`.

Make sure every folder has an empty file called `__init__.py` in it so Python recognizes them as modules.

Create `backend/app/config.py`:

```
# backend/app/config.py

from pydantic_settings import BaseSettings, SettingsConfigDict
from typing import Optional

# BaseSettings automatically reads from our .env file!
class Settings(BaseSettings):
    # App Settings
    PROJECT_NAME: str = "LeaseGPT API"
    ENVIRONMENT: str = "development"
    API_SECRET_KEY: str = "change_me_in_prod"

    # Database Settings
    DATABASE_URL: str
    REDIS_URL: str

    # AI Settings
    OLLAMA_BASE_URL: str = "http://localhost:11434"
    LLM_MODEL: str = "llama3.1:8b"
    EMBEDDING_MODEL: str = "nomic-embed-text"

    # We define the dimension of our vector based on the model we use.
    # nomic-embed-text outputs a list of 768 numbers.
    VECTOR_DIMENSION: int = 768

    # This tells Pydantic to look for a file named .env in the parent directory
    model_config = SettingsConfigDict(env_file="../.env", extra="ignore")

# Create a global instance of settings to use throughout our app
settings = Settings()
```


What just happened? We created a Configuration class. Pydantic looks at our `.env` file, grabs the values, ensures they are the right type (e.g. `VECTOR_DIMENSION` must be an integer), and makes them available securely to our Python code.

Database Connection

Create `backend/app/db/session.py` :

```
# backend/app/db/session.py

from sqlalchemy.ext.asyncio import create_async_engine, async_sessionmaker, AsyncSession
from app.config import settings

# 1. Create the Engine
# The engine manages the connection pool to the database.
# We use create_async_engine because we want our server to be able to handle
# thousands of requests simultaneously without waiting for the database to respond.
engine = create_async_engine(
    settings.DATABASE_URL,
    echo=False, # Set to True to see all SQL queries printed in the terminal
    future=True
)

# 2. Create the Session Factory
# A session is a single "conversation" with the database.
# async_sessionmaker generates new sessions for us when we need them.
AsyncSessionLocal = async_sessionmaker(
    engine,
    class_=AsyncSession,
    expire_on_commit=False # Prevents SQLAlchemy from wiping our variables after saving
)

# 3. Dependency Injection Function
# We will use this function in FastAPI routes to get a database connection.
# The 'yield' keyword makes this a generator. It opens the connection, gives it to the
# and then automatically closes it when the route is finished.
async def get_db():
    async with AsyncSessionLocal() as session:
        yield session
```

What just happened? We configured SQLAlchemy to connect to our PostgreSQL database asynchronously. Think of the `engine` as the telephone line to the database, and the `session` as an active phone call.

Defining our Database Tables (Models)

Instead of writing raw SQL code (like `CREATE TABLE users ...`), we write Python classes. SQLAlchemy translates these classes into SQL for us. This is called an ORM (Object Relational Mapper).

Create a base file for our models: `backend/app/models/base.py` :

```
# backend/app/models/base.py

from sqlalchemy.orm import declarative_base

# All our future database models will inherit from this Base class
Base = declarative_base()
```

Now let's define the tables for our Leases. Create `backend/app/models/lease.py` :

```
# backend/app/models/lease.py

import uuid
from datetime import datetime
from sqlalchemy import Column, String, DateTime, ForeignKey, JSON
from sqlalchemy.dialects.postgresql import UUID, JSONB
from sqlalchemy.orm import relationship
from pgvector.sqlalchemy import Vector
from app.models.base import Base
from app.config import settings

class LeaseDocument(Base):
    """
    Represents an uploaded lease PDF in our database.
    """
    __tablename__ = "lease_documents" # The actual name of the table in Postgres

    # We use UUIDs instead of standard 1,2,3 IDs for better security
    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    filename = Column(String, nullable=False)
    s3_url = Column(String, nullable=True) # Where the physical file is stored in AWS
    status = Column(String, default="processing") # "processing", "completed", "failed"

    # Store extra flexible data (like page count, file size) in a JSON column
    metadata_ = Column("metadata", JSONB, default={})

    created_at = Column(DateTime, default=datetime.utcnow)

    # Establish a relationship with the chunks (One Lease has Many Chunks)
    chunks = relationship("LeaseChunk", back_populates="document", cascade="all, delete")

class LeaseChunk(Base):
    """
    When we process a 50-page lease, we break it into small "chunks" (e.g. 500 words each).
    We turn each chunk into a Vector Embedding.
    """
    __tablename__ = "lease_chunks"

    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)

    # Link back to the parent LeaseDocument
    document_id = Column(UUID(as_uuid=True), ForeignKey("lease_documents.id"), nullable=False)

    # The actual text of this paragraph/chunk
    text_content = Column(String, nullable=False)

    # THE MAGIC HAPPENS HERE: This column stores our 768-dimension vector embedding!
    embedding = Column(Vector(settings.VECTOR_DIMENSION), nullable=False)

    # Extra data like {"page_number": 4, "section_title": "Pets"}
    chunk_metadata = Column(JSONB, default={})

    # Link back to the parent Document
    document = relationship("LeaseDocument", back_populates="chunks")
```

What just happened? We created two tables. `LeaseDocument` tracks the overall uploaded file. Because an LLM can't process an entire 50-page lease at once, we break the lease into smaller paragraphs called `LeaseChunks`. We use `pgvector`'s `Vector` column to store the mathematical representation of that paragraph so we can search it semantically later.

Database Migrations with Alembic

We have Python classes, but our PostgreSQL database doesn't know about them yet! We need to push these changes to the database. We use **Alembic** to generate migration scripts.

In your `backend` folder, run:

```
alembic init alembic
```

This creates an `alembic` folder and an `alembic.ini` file.

Open `backend/alembic.ini` and find the `sqlalchemy.url` line. Change it to this (or comment it out, since we will set it in python):

```
# backend/alembic.ini (around line 58)
# We comment this out because we want to use our settings.DATABASE_URL from config.py
# sqlalchemy.url = driver://user:pass@localhost/dbname
```

Now open `backend/alembic/env.py` and modify it heavily to look like this. This connects Alembic to our asynchronous SQLAlchemy setup:

```
# backend/alembic/env.py

import asyncio
from logging.config import fileConfig
from sqlalchemy import pool
from sqlalchemy.engine import Connection
from sqlalchemy.ext.asyncio import async_engine_from_config
from alembic import context

# Import our settings and base models
from app.config import settings
from app.models.base import Base
# IMPORTANT: You MUST import your models here so Alembic can see them!
import app.models.lease

config = context.config

if config.config_file_name is not None:
    fileConfig(config.config_file_name)

# Tell Alembic where our metadata is
target_metadata = Base.metadata

def run_migrations_offline() → None:
    """Run migrations in 'offline' mode."""
    context.configure(
        url=settings.DATABASE_URL,
        target_metadata=target_metadata,
        literal_binds=True,
        dialect_opts={"paramstyle": "named"},
    )
    with context.begin_transaction():
        context.run_migrations()

def do_run_migrations(connection: Connection) → None:
    # This is required for pgvector to work with Alembic!
    context.configure(connection=connection, target_metadata=target_metadata)
    with context.begin_transaction():
        context.run_migrations()

async def run_async_migrations() → None:
    """In this scenario we need to create an Engine
    and associate a connection with the context."""
    configuration = config.get_section(config.config_ini_section, {})
    configuration["sqlalchemy.url"] = settings.DATABASE_URL

    connectable = async_engine_from_config(
        configuration,
        prefix="sqlalchemy.",
        poolclass=pool.NullPool,
    )

    async with connectable.connect() as connection:
        await connection.run_sync(do_run_migrations)

    await connectable.dispose()
```

```
def run_migrations_online() → None:
    """Run migrations in 'online' mode."""
    asyncio.run(run_async_migrations())

if context.is_offline_mode():
    run_migrations_offline()
else:
    run_migrations_online()
```

Now, let's create our first migration! In the terminal (ensure your docker database is running and your venv is activated):

```
# This creates a python script with the SQL commands needed to build our tables
alembic revision --autogenerate -m "Initial tables"

# This actually runs the script against the database
alembic upgrade head
```

[!SUCCESS]

Congratulations! You have successfully configured a robust, asynchronous, vector-ready database layer.

In **Part 2**, we will build out the FastAPI server, create our API endpoints, and begin integrating the LlamaIndex RAG pipeline to actually chunk text and save vectors into this database!

LeaseGPT Tutorial - Part 2: Backend API & The RAG Pipeline

Welcome to Part 2 of the LeaseGPT tutorial! In Part 1, we set up our database and project structure. Now, we're going to build the brain and the nervous system of our application.

We will build the **Backend API** (how our frontend will talk to our system) and the **RAG Pipeline** (how our system actually reads and understands leases).

Chapter 4: Building the Backend API (FastAPI)

Let's start by defining some key concepts before we write code.

Beginner Concepts

- **What is FastAPI?** FastAPI is a modern web framework for Python. Think of a web framework as a toolkit that makes it easy to build websites or APIs. FastAPI is special because it is extremely fast, automatically generates documentation for you, and uses modern Python features like "async" (meaning it can handle multiple requests at the same time without waiting).
- **What is a REST API?** API stands for Application Programming Interface. A REST API is a standard way for two computers to talk to each other over the internet using HTTP (the same protocol your web browser uses). Our React frontend will send "requests" to our FastAPI backend, and our backend will send "responses" back.
- **What are API routes/endpoints?** A route (or endpoint) is a specific URL that triggers a specific function in your code. For example, sending a request to `http://localhost:8000/health` might trigger a function that checks if the server is running.
- **What is CORS?** CORS stands for Cross-Origin Resource Sharing. By default, web browsers block web pages from making requests to a different domain or port for security reasons. Since our frontend will run on port 3000 and our backend on port 8000, we need to configure CORS to explicitly tell the browser, "It's okay, let port 3000 talk to port 8000."
- **What is Dependency Injection?** Imagine building a house. Instead of every worker building their own hammer, you have a tool shed where workers request a hammer when they need one. Dependency injection in FastAPI works similarly. Instead of every route opening its own

database connection, routes simply state "I need a database connection," and FastAPI provides (injects) it automatically.

- **What is Middleware?** Middleware is code that runs *between* receiving a request and processing it, or *between* processing a request and sending the response. Think of it like a security checkpoint at a building. Every visitor (request) must pass through the checkpoint (middleware) before they can visit an office (route).

1. `backend/app/main.py` — The Application Factory

This is the entry point of our backend. It sets up the FastAPI application, configures middleware, and registers our routes.


```
# backend/app/main.py

from contextlib import asynccontextmanager # Helps us manage startup and shutdown code
from fastapi import FastAPI # The main FastAPI class
from fastapi.middleware.cors import CORSMiddleware # Middleware for handling CORS

# We'll import our routers later when we create them
# from app.api.routes import health, upload, leases

@asynccontextmanager
async def lifespan(app: FastAPI):
    """
    The lifespan context manager.
    Code before the 'yield' runs when the application starts up.
    Code after the 'yield' runs when the application shuts down.
    """
    # STARTUP: We might want to connect to the database or Redis here
    print("Starting up LeaseGPT backend... ")

    # We pause execution here and let the FastAPI app run
    yield

    # SHUTDOWN: We close connections gracefully here
    print("Shutting down LeaseGPT backend... ")

# Create the actual FastAPI application instance
app = FastAPI(
    title="LeaseGPT API",
    description="API for uploading leases and chatting with them via RAG",
    version="1.0.0",
    lifespan=lifespan # Attach our lifespan manager
)

# Configure CORS (Cross-Origin Resource Sharing)
app.add_middleware(
    CORSMiddleware,
    # In development, we allow requests from the React frontend running on port 3000
    allow_origins=["http://localhost:3000"],
    allow_credentials=True,
    allow_methods=["*"], # Allow all HTTP methods (GET, POST, DELETE, etc.)
    allow_headers=["*"], # Allow all headers
)

# Register our routes (we will uncomment these as we build them)
# app.include_router(health.router, prefix="/health", tags=["Health"])
# app.include_router(upload.router, prefix="/api/leases", tags=["Upload"])
# app.include_router(leases.router, prefix="/api/leases", tags=["Leases"])

@app.get("/")
async def root():
    """A simple default route just to show the server is alive."""
    return {"message": "Welcome to the LeaseGPT API!"}
```

NOTE

****What just happened?**** We created the foundation of our web server. We used a

`lifespan` function which is a clean way to handle startup (like connecting to a database) and shutdown (like closing connections). The `yield` keyword is where the app actually runs. We also added CORS middleware so our future React frontend won't get blocked by the browser.

2. `backend/app/api/dependencies.py` — Dependency Injection

Here we define the things our routes will need, primarily database and Redis connections.

```
# backend/app/api/dependencies.py

from typing import AsyncGenerator
from sqlalchemy.ext.asyncio import AsyncSession
# Assume we have a sessionmaker set up in our database setup file
# from app.db.session import async_session_maker
# from app.core.config import get_settings, Settings
# import redis.asyncio as redis

# async def get_settings() → Settings:
#     """Returns the application settings."""
#     return get_settings()

async def get_db() → AsyncGenerator[AsyncSession, None]:
    """
    Dependency to get a database session.
    Using 'yield' ensures the session is closed after the request is finished.
    """
    # We will create an async session from our session factory
    # async with async_session_maker() as session:
    #     yield session
    pass # Placeholder for now

async def get_redis() → AsyncGenerator: # AsyncGenerator[redis.Redis, None]
    """
    Dependency to get a Redis connection.
    """
    # client = redis.from_url("redis://localhost:6379/0")
    # try:
    #     yield client
    # finally:
    #     await client.aclose()
    pass # Placeholder for now
```

✓ TIP

****Why `yield` instead of `return`?**** When you `return` something, the function finishes immediately. When you `yield`, the function pauses, hands the database session to the route, and waits. Once the route finishes its job (whether it succeeded or crashed), the function resumes after the `yield` and safely closes the database connection. It prevents memory leaks!

3. backend/app/api/middleware.py — Custom Middleware

Let's add some middleware to track requests and prevent abuse (rate limiting).

```
# backend/app/api/middleware.py

import time
import logging
from fastapi import Request
from starlette.middleware.base import BaseHTTPMiddleware
from starlette.responses import JSONResponse

# Setup a basic logger
logger = logging.getLogger(__name__)

class RequestLoggingMiddleware(BaseHTTPMiddleware):
    """
    Middleware that logs every incoming request and how long it took to process.
    """
    async def dispatch(self, request: Request, call_next):
        # Record the start time
        start_time = time.time()

        # Log the incoming request method (GET, POST) and URL path
        logger.info(f"Incoming Request: {request.method} {request.url.path}")

        # Pass the request to the next step (which is eventually the route handler)
        response = await call_next(request)

        # Calculate how long it took
        process_time = time.time() - start_time

        # Log the completion and time taken
        logger.info(f"Completed Request: {request.method} {request.url.path} in {process_time}")

        # Add a custom header to the response showing the processing time
        response.headers["X-Process-Time"] = str(process_time)
        return response

class SimpleRateLimitMiddleware(BaseHTTPMiddleware):
    """
    A very simple rate limiter using a sliding window.
    (In a real app, you would use Redis for this across multiple servers)
    """
    async def dispatch(self, request: Request, call_next):
        # We'll pretend we are checking Redis to see if this user has made too many requests
        # If they did, we would return a 429 Too Many Requests response:
        # return JSONResponse(status_code=429, content={"detail": "Too many requests"})

        # Otherwise, proceed as normal
        return await call_next(request)
```

NOTE

****What just happened?**** The `RequestLoggingMiddleware` acts like a stopwatch. It starts

the timer when a request arrives, calls `call_next(request)` to let the rest of the application do its work, and then stops the timer when the response comes back. It logs the time and even adds an `X-Process-Time` header to the final response.

4. `backend/app/api/routes/health.py` — Health Checks

Health checks are crucial for modern deployment systems like Kubernetes or Docker Compose to know if your app is alive.

```
# backend/app/api/routes/health.py

from fastapi import APIRouter, Depends
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import text
# from app.api.dependencies import get_db

router = APIRouter()

@router.get("/")
async def liveness_check():
    """
    Liveness probe.
    If this endpoint returns a 200 OK, the web server is running.
    """
    return {"status": "alive"}

@router.get("/ready")
async def readiness_check(): # Add: session: AsyncSession = Depends(get_db)
    """
    Readiness probe.
    Checks if the application is ready to accept traffic (e.g., database is connected).
    """
    try:
        # We would try to run a simple query to ensure the DB is connected
        # await session.execute(text("SELECT 1"))
        return {"status": "ready", "database": "connected"}
    except Exception as e:
        # If the query fails, the app is not ready to serve traffic
        from fastapi import HTTPException
        raise HTTPException(status_code=503, detail="Database unavailable")
```

5. `backend/app/api/routes/upload.py` — File Uploads

Here we handle users uploading their lease PDFs.

```
# backend/app/api/routes/upload.py

import uuid
from fastapi import APIRouter, File, UploadFile, HTTPException, Depends
from typing import Dict

# from app.services.upload_service import process_upload
# from app.api.dependencies import get_db
# from sqlalchemy.ext.asyncio import AsyncSession

router = APIRouter()

# Define allowed file extensions
ALLOWED_EXTENSIONS = {"pdf", "png", "jpg", "jpeg", "tiff"}
# Max file size: 20MB
MAX_FILE_SIZE = 20 * 1024 * 1024

@router.post("/")
async def upload_lease(
    file: UploadFile = File(...), # 'File(...)' tells FastAPI this is a multipart form
    # db: AsyncSession = Depends(get_db)
) → Dict[str, str]:
    """
    Endpoint to upload a lease document.
    """
    # 1. Validate the file extension
    filename = file.filename
    ext = filename.split(".")[-1].lower() if "." in filename else ""
    if ext not in ALLOWED_EXTENSIONS:
        raise HTTPException(status_code=400, detail=f"Unsupported file type. Allowed: {")

    # 2. Read the file into memory (we do this to check size, but be careful with HUGE
    contents = await file.read()

    # 3. Validate file size
    if len(contents) > MAX_FILE_SIZE:
        raise HTTPException(status_code=413, detail="File too large. Maximum size is 20MB")

    # 4. Generate a unique ID for this lease
    lease_id = str(uuid.uuid4())

    # 5. In a real app, we'd pass this to a service to upload to S3 and save to DB
    # await process_upload(lease_id, filename, contents, db)

    # For now, just return success
    return {
        "lease_id": lease_id,
        "message": "File uploaded successfully. Processing started.",
        "status": "processing"
    }
```

IMPORTANT

****What is a multipart upload?**** Normally, HTTP requests send text (like JSON).

`multipart/form-data` is a special way to send large binary files (like PDFs or images) over

HTTP by breaking them into parts. FastAPI's `UploadFile` handles all the complexity of this for us.

6. Services and Utils (The Business Logic)

We won't write every line of these, but it's important to understand the structure.

Why a Service Layer? Routes should only handle web stuff (receiving requests, validating inputs, returning responses). The actual work (saving to the database, calling S3) should live in "Services". This makes your code reusable.

`backend/app/services/upload_service.py` (Conceptual)

This file would take the raw file bytes, use the `s3.py` utility to upload the file to Amazon S3 (or a local folder in development), and then create a new record in our PostgreSQL database with a status of "processing".

`backend/app/utils/s3.py` (Conceptual)

This file uses the `boto3` Python library to communicate with AWS. Think of S3 as an infinite hard drive in the cloud. We store the massive PDF files there, and only store a *link* to that file in our PostgreSQL database.

Chapter 5: Document Processing (Text Extraction)

Now that we have the file uploaded, we need to read the text inside it.

Beginner Concepts

- **What is OCR?** Optical Character Recognition. If you scan a physical piece of paper into a PDF, the computer just sees an image (a bunch of pixels). It doesn't know there are words there. OCR is AI that looks at the pixels and types out the letters it sees.
- **What is AWS Textract?** It's a powerful OCR service provided by Amazon. It's smart enough to understand tables, forms, and signatures.
- **What is PyMuPDF?** It's a fast Python library. If a PDF was created digitally (e.g., exported from Microsoft Word), the text is already embedded inside it. We don't need OCR; PyMuPDF can just rip the text right out instantly.

1. `backend/app/services/document_processor.py`

This service decides how to read the file.

```
# backend/app/services/document_processor.py

import fitz # This is PyMuPDF (confusing name, I know!)
import io
from typing import List, Dict

# from app.utils.textract import run_textract_ocr

async def process_document(file_bytes: bytes, filename: str) → List[Dict]:
    """
    Takes a file, determines if it needs OCR, and extracts the text page by page.
    Returns a list of dictionaries, one for each page.
    """
    extension = filename.split(".")[1].lower() if "." in filename else ""

    extracted_pages = []

    # If it's an image, we MUST use OCR
    if extension in ["jpg", "jpeg", "png", "tiff"]:
        print("Image detected. Running OCR... ")
        # extracted_pages = await run_textract_ocr(file_bytes)
        # Mocking the result for this tutorial
        extracted_pages = [{"page_num": 1, "text": "This is text extracted via OCR from"}
        return extracted_pages

    # If it's a PDF, check if it has text layer
    if extension == "pdf":
        print("PDF detected. Analyzing... ")
        # Open the PDF using PyMuPDF from memory
        pdf_document = fitz.open("pdf", file_bytes)

        needs_ocr = False

        # Let's check the first page to see if it has text
        first_page = pdf_document[0]
        text_content = first_page.get_text()

        # If the page is basically empty of text, it's probably a scanned image inside
        if len(text_content.strip()) < 50:
            needs_ocr = True

        if needs_ocr:
            print("Scanned PDF detected. Running OCR... ")
            # extracted_pages = await run_textract_ocr(file_bytes)
        else:
            print("Digital PDF detected. Extracting text directly... ")
            # Loop through all pages and extract text
            for page_num in range(len(pdf_document)):
                page = pdf_document[page_num]
                extracted_pages.append({
                    "page_num": page_num + 1, # 1-indexed pages
                    "text": page.get_text()
                })

        pdf_document.close()
        return extracted_pages
```

✓ TIP

****Why do we check if it's a scanned PDF?**** AWS Textract costs money per page. PyMuPDF is completely free and runs locally. By checking if the PDF already contains digital text, we save time and money!

Chapter 6: The RAG Pipeline (Chunking, Embedding, Retrieval)

This is the brain of LeaseGPT. RAG stands for **Retrieval-Augmented Generation**.

Normally, an LLM (like ChatGPT) only knows what it was trained on months ago. It doesn't know about *your* specific lease. RAG solves this. We **Retrieve** relevant parts of your lease, and give them to the LLM to **Augment** its knowledge, so it can **Generate** an accurate answer.

Beginner Concepts

- **What is Chunking?** Imagine a 100-page lease. If a user asks "What is the pet policy?", we shouldn't feed all 100 pages to the AI. AIs have limits on how much they can read at once (context window), and it's expensive. Instead, we break the lease up into small "chunks" (paragraphs). We then search for the *most relevant* chunks and only feed those to the AI.
- **What is a Vector Embedding?** This is the magic of modern AI search. An embedding model takes text (e.g., "The rent is \$2000/month") and turns it into a giant array of numbers, like `[0.12, -0.45, 0.88, ...]`. These numbers represent the *meaning* of the text in mathematical space. Sentences with similar meanings will have similar numbers, even if they use completely different words!
- **Semantic Search vs Keyword Search:**
 - **Keyword:** Searching for exactly the word "rent". Misses "monthly payment".
 - **Semantic:** Comparing those arrays of numbers (vectors). Searching for "how much do I owe" will mathematically match the chunk about "monthly rent" because their meanings are close.
- **Reciprocal Rank Fusion (RRF):** We actually want *both* searches. Keyword search is great for exact names or section numbers. Semantic search is great for concepts. RRF is a fancy math formula that takes the top results from both searches and blends them into one super-accurate ranked list.

1. `backend/app/rag/indexer.py` — Breaking it Down

This file takes the extracted text, chunks it, embeds it, and saves it to PostgreSQL.


```
# backend/app/rag/indexer.py

from typing import List, Dict
import uuid
# We use LlamaIndex's built-in splitter because it's very smart about not breaking sent
from llama_index.core.node_parser import SentenceSplitter

# In a real app, we'd import our embedding model (e.g., Ollama) and database session he

async def index_document(lease_id: str, extracted_pages: List[Dict]):
    """
    Takes pages of text, chunks them, creates embeddings, and saves them to the DB.
    """
    print(f"Indexing lease {lease_id} ... ")

    # 1. Initialize the sentence splitter
    # chunk_size=512: Target size is roughly 512 tokens (words/pieces of words)
    # chunk_overlap=50: Leave a 50-token overlap between chunks so context isn't lost a
    splitter = SentenceSplitter(chunk_size=512, chunk_overlap=50)

    all_chunks_data = []

    # 2. Iterate through pages and chunk the text
    for page in extracted_pages:
        page_num = page["page_num"]
        text = page["text"]

        # The splitter breaks the page text into smaller chunks
        text_chunks = splitter.split_text(text)

        for chunk_text in text_chunks:
            # 3. Generate a vector embedding for this chunk
            # embedding = await get_embedding_model().get_text_embedding(chunk_text)
            embedding = [0.0] * 768 # Mocking a 768-dimensional vector (what nomic-embed

            # 4. Prepare the data to be saved in the database
            chunk_data = {
                "id": str(uuid.uuid4()),
                "lease_id": lease_id,
                "page_number": page_num,
                "text_content": chunk_text,
                "embedding": embedding
            }
            all_chunks_data.append(chunk_data)

    # 5. Save all_chunks_data to the PostgreSQL database
    # This involves creating LeaseChunk models and saving them via SQLAlchemy
    print(f"Created {len(all_chunks_data)} chunks. Saving to DB ... ")
    # await db_session.add_all(chunk_models)
    # await db_session.commit()

    print("Indexing complete!")
```

◆ WARNING

****Why do we need overlap?**** If Chunk 1 ends with "The tenant is responsible for paying"

and Chunk 2 starts with "the water bill," searching for "water bill responsibility" might fail because the concept was cut in half! Overlap ensures context bleeds between chunks.

2. `backend/app/rag/retriever.py` — Finding the Answers

When a user asks a question, this file searches the database for the most relevant chunks.

```
# backend/app/rag/retriever.py

from sqlalchemy import text
from sqlalchemy.ext.asyncio import AsyncSession
from typing import List, Dict

async def hybrid_search(db: AsyncSession, lease_id: str, query_text: str, limit: int = 10)
    """
    Performs a hybrid search (Semantic + Keyword) using Reciprocal Rank Fusion (RRF).
    """

    # 1. Convert the user's question into a vector embedding
    # query_embedding = await get_embedding_model().get_text_embedding(query_text)
    query_embedding_str = "[0.1, 0.2, ...]" # In reality, we format the list of floats

    # 2. The massive SQL Query
    # This uses Common Table Expressions (CTEs - the 'WITH' blocks) to break down the query
    rrf_query = text(f"""
    WITH
    -- Step A: Semantic Vector Search
    -- We use pgvector's <=> operator which calculates Cosine Distance (how different two vectors are)
    -- We order by distance ascending (smaller distance = more similar).
    semantic_search AS (
        SELECT id, text_content, page_number,
            ROW_NUMBER() OVER(ORDER BY embedding <=> :query_embedding) AS rank
        FROM lease_chunks
        WHERE lease_id = :lease_id
        LIMIT :limit
    ),
    -- Step B: Keyword Full-Text Search
    -- We use PostgreSQL's built-in full text search (to_tsvector and plainto_tsquery).
    -- We rank results using ts_rank based on keyword frequency.
    keyword_search AS (
        SELECT id, text_content, page_number,
            ROW_NUMBER() OVER(ORDER BY ts_rank(to_tsvector('english', text_content),
                plainto_tsquery('english', :query_text))
        FROM lease_chunks
        WHERE lease_id = :lease_id
        AND to_tsvector('english', text_content) @@ plainto_tsquery('english', :query_text)
        LIMIT :limit
    )
    -- Step C: Reciprocal Rank Fusion (RRF)
    -- We combine the results from semantic and keyword searches.
    -- The formula is 1 / (constant + rank). A common constant is 60.
    -- If a chunk scores high in BOTH searches, its RRF score is massive!
    SELECT
        COALESCE(s.id, k.id) as chunk_id,
        COALESCE(s.text_content, k.text_content) as text_content,
        COALESCE(s.page_number, k.page_number) as page_number,
        (COALESCE(1.0 / (60 + s.rank), 0.0) + COALESCE(1.0 / (60 + k.rank), 0.0)) as rrf_score
    FROM semantic_search s
    FULL OUTER JOIN keyword_search k ON s.id = k.id
    ORDER BY rrf_score DESC
    LIMIT :limit
    """)
```

```
# 3. Execute the query
# result = await db.execute(rrf_query, {
#     "query_embedding": query_embedding_str,
#     "query_text": query_text,
#     "lease_id": lease_id,
#     "limit": limit
# })

# chunks = result.fetchall()
# return [{"id": row.chunk_id, "text": row.text_content, "page": row.page_number} f

return [{"text": "Mocked resulting chunk about rent.", "page": 4}]
```

💡 NOTE

****What just happened in that SQL?**** 1. We found the top 5 chunks that matched the **meaning** (semantic). 2. We found the top 5 chunks that contained the **exact words** (keyword). 3. We gave every chunk a score based on its rank in those lists (Rank 1 gets a high score, Rank 5 gets a lower score). 4. We added the scores together. If a chunk was #1 in semantic AND #1 in keyword, it wins big! We return those top winners.

3. `backend/app/rag/prompts.py` — Prompt Engineering

Finally, we take those relevant chunks and hand them to the AI to answer the question.

```
# backend/app/rag/prompts.py

# A System Prompt tells the AI who it is and what rules it must follow.
# This is critical for preventing hallucinations (the AI making things up).

QA_SYSTEM_PROMPT = """
You are a highly accurate legal assistant specializing in residential and commercial law.
Your job is to answer the user's question using ONLY the context provided below.

Context blocks (excerpts from the lease) are provided with their page numbers.

RULES:
1. Do not use outside knowledge. If the answer is not in the context, say "I cannot find the answer."
2. Always cite your sources using the page number provided in the context. Format citations as (Page X).
3. Be concise but thorough.
4. If there are conflicting statements in the lease, point them out.

CONTEXT:
{context_string}
"""

def build_qa_prompt(user_question: str, retrieved_chunks: List[Dict]) → str:
    """
    Takes the chunks we retrieved from the database and formats them into a single string
    that the LLM can understand, then combines it with the system prompt and the user's question.
    """
    context_string = ""
    for chunk in retrieved_chunks:
        context_string += f"---\n[Page {chunk['page']}] \n{chunk['text']}\n"

    system_prompt_filled = QA_SYSTEM_PROMPT.format(context_string=context_string)

    # We would pass this system_prompt_filled AND the user_question to the LLM (like OpenAI)
    return system_prompt_filled + user_question
```

Wrapping Up Part 2

We now have a backend that can accept a file, extract its text (using OCR or directly), chunk that text, save vector embeddings to PostgreSQL, and perform lightning-fast hybrid search to find answers!

In Part 3, we will connect all of this to a beautiful React frontend.

LeaseGPT Tutorial - Part 3: The AI Agent and Real-Time Chat

Welcome to Part 3 of the LeaseGPT tutorial! In Parts 1 and 2, we set up our backend architecture, built the document processing pipeline to read PDFs, and created our Retrieval-Augmented Generation (RAG) system using LlamaIndex and pgvector.

Now, it's time to build the **Brain** (the AI Agent) and the **Mouth/Ears** (Real-Time WebSockets) of our application. We want users to be able to ask questions and watch the AI type out the answer in real-time, just like ChatGPT.

Chapter 7: The AI Agent (LangGraph)

What is an AI Agent?

Think of a traditional computer program as a fast-food worker strictly following a recipe: *Put bun, add burger, add cheese, add bun*. It does exactly what you say, in the exact order, every single time.

An **AI Agent** is more like a **Detective**. When you ask a detective a question, they don't just blurt out an answer. They:

1. **Analyze** what you're asking.
2. **Gather Evidence** (look through files).
3. **Evaluate** if the evidence is good enough (if not, they search again).
4. **Write a Report** based *only* on the evidence they found.

Instead of a hardcoded recipe, an agent uses a Large Language Model (LLM) to make *decisions* about what to do next.

What is a Graph?

To organize our detective's workflow, we use **LangGraph**. A "graph" in computer science is just a map of connected points.

* **Nodes (Steps)**: The actual tasks (e.g., "Analyze", "Retrieve", "Generate Answer"). In our code, these are just Python functions.

* **Edges (Connections)**: The arrows pointing from one step to the next.

* **Conditional Edges (Decisions):** Forks in the road. (e.g., *Is the evidence good enough? Yes -> Generate Answer. No -> Go back and Retrieve again.*)

What is State?

As the detective moves from step to step, they carry a **Case File**. In LangGraph, this is called **State**. It's an object (a bundle of data) that gets passed to a node. The node reads the state, updates it with new information, and passes it to the next node.

Here is the visual map of the Graph we are about to build:

```
graph TD
    Start(("Start"))
    Analyze["Analyze Query\n(Extract keywords)"]
    Retrieve["Retrieve Context\n(Search Database)"]
    Check["Check Relevance\n(Is it good enough?)"]
    Generate["Generate Answer\n(Write response)"]
    Format["Format Citations\n(Add page numbers)"]
    End(("End"))

    Start --> Analyze
    Analyze --> Retrieve
    Retrieve --> Check
    Check -->|"Relevant (Score ≥ 0.5)"| Generate
    Check -->|"Not Relevant (Retry)"| Retrieve
    Generate --> Format
    Format --> End
```

1. The LangGraph Q&A Agent

Let's write the code for our detective. Create a new file at `backend/app/rag/agent.py`.

```
# backend/app/rag/agent.py

import json
from typing import Optional, List, Dict, Any, AsyncGenerator
from uuid import UUID
from pydantic import BaseModel, Field
from langgraph.graph import StateGraph, END
# We use LlamaIndex for the actual LLM calls
from llama_index.core.llms import ChatMessage, MessageRole
from llama_index.llms.ollama import Ollama

# These are placeholder imports for the services we built in Part 2
# from app.rag.retriever import hybrid_search
# from app.core.config import settings

# -----
# 1. DEFINE THE STATE (The Detective's Case File)
# -----

class ChunkResult(BaseModel):
    """Represents a piece of text found in the database."""
    text: str
    page_num: int
    section: str
    score: float

class Citation(BaseModel):
    """Represents a source reference for the user."""
    page_num: int
    snippet: str

class QAState(BaseModel):
    """
    This is our State. It holds all the information as we move
    through the graph. Notice how many fields are Optional?
    That's because they start empty and get filled in step-by-step.
    """
    lease_id: UUID
    query: str
    # What the LLM thinks the user is asking
    query_analysis: Optional[Dict[str, Any]] = None
    # The text chunks we pull from Postgres
    retrieved_chunks: List[ChunkResult] = Field(default_factory=list)
    # How good the chunks are (0.0 to 1.0)
    relevance_score: Optional[float] = None
    # The final written response
    answer: Optional[str] = None
    # Where we found the info
    citations: List[Citation] = Field(default_factory=list)
    # How many times we've looped back to retry
    retry_count: int = 0
    # Any errors that happen
    error: Optional[str] = None

# -----
# 2. INITIALIZE THE LLM
# -----
# We use Ollama running locally.
```



```

llm = Ollama(model="llama3", request_timeout=120.0)

# -----
# 3. DEFINE THE NODES (The Steps in the Workflow)
# -----

async def analyze_query(state: QASState) → QASState:
    """
    Node 1: Look at the user's question and extract key terms.
    """
    # Create a prompt telling the LLM to act like a keyword extractor
    prompt = f"""
    Analyze the following query about a lease agreement.
    Extract the main entities and keywords for searching.
    Return ONLY valid JSON with a 'keywords' list.
    Query: {state.query}
    """

    # Send the prompt to the LLM
    response = await llm.acomplete(prompt)

    try:
        # Try to parse the LLM's text output into a Python dictionary
        analysis = json.loads(response.text)
        state.query_analysis = analysis
    except:
        # If the LLM didn't return perfect JSON, just fallback to the raw query
        state.query_analysis = {"keywords": [state.query]}

    return state

async def retrieve_context(state: QASState) → QASState:
    """
    Node 2: Search the database using the keywords.
    """
    # Grab the keywords we generated in the last step
    keywords = state.query_analysis.get("keywords", [state.query])
    search_term = " ".join(keywords)

    # Call our hybrid search function (from Part 2)
    # Mocked here for the tutorial structure
    # chunks = await hybrid_search(state.lease_id, search_term)

    # For demonstration, we'll pretend we got these chunks
    mock_chunks = [
        ChunkResult(text="Rent is $2000 per month.", page_num=1, section="Rent", score=0.8),
        ChunkResult(text="Pets are not allowed.", page_num=2, section="Pets", score=0.8)
    ]

    state.retrieved_chunks = mock_chunks
    # We increment the retry count every time we hit this node
    state.retry_count += 1

    return state

async def check_relevance(state: QASState) → QASState:
    """
    Node 3: LLM-as-a-Judge. Are these chunks actually helpful?

```

```

"""
context_str = "\n".join([c.text for c in state.retrieved_chunks])

prompt = f"""
Given the user's question and the retrieved lease context, rate how relevant
the context is for answering the question on a scale of 0.0 to 1.0.
Return ONLY a number.
Question: {state.query}
Context: {context_str}
"""

response = await llm.acomplete(prompt)

try:
    # Parse the number out of the text
    score = float(response.text.strip())
    state.relevance_score = score
except:
    # If we can't parse it, assume it's good enough to proceed
    state.relevance_score = 1.0

return state

async def generate_answer(state: QASState) → QASState:
    """
    Node 4: Write the final answer using the context.
    """
    # Combine all our chunks into a big string of context
    context_str = ""
    for idx, chunk in enumerate(state.retrieved_chunks):
        # We add [Page X] tags so the LLM knows where the info came from
        context_str += f"[Page {chunk.page_num}] {chunk.text}\n"

    prompt = f"""
    You are a helpful real estate assistant. Answer the user's question using ONLY
    the provided context from their lease agreement.
    Whenever you state a fact, cite the page number like this: (Page X).
    If the context does not contain the answer, say "I cannot find this in the lease."

    Context:
    {context_str}

    Question: {state.query}
    """

    # Get the final answer from the LLM
    response = await llm.acomplete(prompt)
    state.answer = response.text

    return state

async def format_citations(state: QASState) → QASState:
    """
    Node 5: Clean up the data for the frontend.
    """
    # For every chunk we used, create a Citation object
    # The frontend will use this to show clickable reference links
    state.citations = [

```

```

        Citation(page_num=c.page_num, snippet=c.text)
        for c in state.retrieved_chunks
    ]
    return state

# -----
# 4. CONDITIONAL LOGIC (Forks in the road)
# -----
def should_retry(state: QASState) → str:
    """
    This function doesn't return a state. It returns the NAME of the next node.
    """
    # If the context sucks (score < 0.5) AND we haven't retried too many times...
    if state.relevance_score < 0.5 and state.retry_count < 2:
        return "retrieve_context" # Go back and search again

    # Otherwise, move on to generating the answer
    return "generate_answer"

# -----
# 5. BUILD THE GRAPH
# -----
# Create a new graph that uses our QASState structure
workflow = StateGraph(QASState)

# Add all our Python functions as nodes in the graph
workflow.add_node("analyze_query", analyze_query)
workflow.add_node("retrieve_context", retrieve_context)
workflow.add_node("check_relevance", check_relevance)
workflow.add_node("generate_answer", generate_answer)
workflow.add_node("format_citations", format_citations)

# Define the normal flow (the arrows)
workflow.set_entry_point("analyze_query")
workflow.add_edge("analyze_query", "retrieve_context")
workflow.add_edge("retrieve_context", "check_relevance")

# Add the fork in the road!
# Depending on what `should_retry` returns, route to the next node.
workflow.add_conditional_edges(
    "check_relevance",
    should_retry,
    {
        "retrieve_context": "retrieve_context", # If it returns this string, go here
        "generate_answer": "generate_answer"    # If it returns this string, go here
    }
)

# Finish the flow
workflow.add_edge("generate_answer", "format_citations")
workflow.add_edge("format_citations", END)

# Compile turns this map into an executable program
app = workflow.compile()

# -----
# 6. RUNNER FUNCTION
# -----

```

```

async def run_agent(lease_id: UUID, query: str) → QASState:
    """
    A helper function to kick off the graph.
    """
    # Create the initial empty state
    initial_state = QASState(lease_id=lease_id, query=query)

    # Run the graph (this is an asynchronous call)
    # LangGraph returns a dictionary, so we unpack it back into our Pydantic model
    final_dict = await app.ainvoke(initial_state.dict())

    return QASState(**final_dict)

```

What just happened?

1. **State:** We defined a "Case File" (`QASState`) using Pydantic. It holds the `query` , `chunks` , `answer` , etc.
2. **Nodes:** We wrote five `async` Python functions. Each takes the `state` , does some work (often asking the AI something), and modifies the `state` .
3. **Graph Setup:** We glued it all together using `StateGraph` .
4. **Conditional Logic:** The magic happens in `should_retry` . It evaluates `relevance_score` and tells LangGraph whether to move forward or loop back.

[!NOTE]

Why use LangGraph instead of just calling these functions in a row?

Because graphs give us **loops** and **memory**. If the search fails, the agent can loop back automatically. In complex apps, you can even pause a graph, ask the user a question, and resume it later!

2. The Chat Service

The agent handles the "thinking", but we need a service to handle the "business logic" (saving chat history to the database, checking Redis for cached answers so we don't pay for AI twice, etc.).

Create `backend/app/services/chat_service.py` :

```
# backend/app/services/chat_service.py

import json
import hashlib
from uuid import UUID
# Imagine we have an async Redis client and Database session configured
# from app.db.redis import redis_client
# from app.db.database import get_db

from app.rag.agent import run_agent

async def generate_chat_response(lease_id: UUID, query: str, session_id: str):
    """
    Manages the full lifecycle of a user asking a question.
    """
    # 1. Create a unique fingerprint (hash) for this specific question
    query_hash = hashlib.md5(query.encode()).hexdigest()
    cache_key = f"chat_cache:{lease_id}:{query_hash}"

    # 2. Check Cache (Redis)
    # If a user asks "What is the rent?" twice, we don't want to run the AI twice.
    # cached_answer = await redis_client.get(cache_key)
    # if cached_answer:
    #     return json.loads(cached_answer)

    # 3. If not cached, RUN THE AGENT (The Brain!)
    final_state = await run_agent(lease_id, query)

    # 4. Format the result for the frontend
    response_data = {
        "answer": final_state.answer,
        "citations": [c.dict() for c in final_state.citations]
    }

    # 5. Save to Cache for next time (expires in 1 hour)
    # await redis_client.setex(cache_key, 3600, json.dumps(response_data))

    # 6. Save messages to Postgres Database (User's question + AI's answer)
    # db_session = await get_db()
    # db_session.add(Message(session_id=session_id, role="user", content=query))
    # db_session.add(Message(session_id=session_id, role="assistant", content=final_state.answer))
    # await db_session.commit()

    return response_data
```

What just happened?

This acts as a wrapper around our agent. It takes the query, checks if we already know the answer (Redis caching), runs the agent if we don't, and then saves the conversation to our Postgres database so the user can see their chat history when they refresh the page.

Chapter 8: Real-Time Chat (WebSockets)

What is a WebSocket?

HTTP (Normal Web Traffic): Like sending a letter.

You (the client) put a letter in a mailbox asking a question. You wait. A few days later, the server sends a single letter back with the whole answer. The connection closes.

WebSocket: Like a phone call.

You dial the server. The server picks up. Now, the line is open. You can talk at any time, and the server can talk at any time, instantly. The connection stays open until someone hangs up.

✓ TIP

****Why do we need this?**** LLMs (like ChatGPT) take time to generate text. If we used HTTP, the user would stare at a loading spinner for 10 seconds. With WebSockets, we can ****STREAM**** the answer. The server sends the text back word-by-word (``"The"`, `"rent"`, `"is"`, `"$2000"``) as it is generated, so the user sees it typing out instantly!

The WebSocket Message Flow

Here's how our server and browser will talk to each other:

```
sequenceDiagram
    participant Browser
    participant Server

    Browser->>Server: Connect (ws://localhost/ws/chat/123)
    Server->>Browser: Accept Connection

    Browser->>Server: {"type": "query", "content": "What is the rent?"}

    Note over Server: Agent starts thinking...

    Server->>Browser: {"type": "token", "content": "The "}
    Server->>Browser: {"type": "token", "content": "rent "}
    Server->>Browser: {"type": "token", "content": "is "}
    Server->>Browser: {"type": "token", "content": "$2000."}

    Note over Server: Agent finishes.

    Server->>Browser: {"type": "citations", "data": [{"page": 1, "snippet": "..."}]}
    Server->>Browser: {"type": "done"}
```

1. The WebSocket Endpoint

Create `backend/app/api/routes/chat.py`. This is our FastAPI route that handles the "phone call".

```

# backend/app/api/routes/chat.py

from fastapi import APIRouter, WebSocket, WebSocketDisconnect
from uuid import UUID
import json
import asyncio

# Import our chat service
from app.services.chat_service import generate_chat_response

router = APIRouter()

@router.websocket("/ws/chat/{lease_id}")
async def websocket_chat(websocket: WebSocket, lease_id: UUID):
    """
    This endpoint stays open as long as the user is on the chat page.
    """
    # 1. "Pick up the phone" - Accept the connection from the browser
    await websocket.accept()

    try:
        # 2. Enter a loop to constantly listen for incoming messages
        while True:
            # Wait for the browser to send us text
            raw_message = await websocket.receive_text()

            # Parse the JSON string into a Python dictionary
            data = json.loads(raw_message)

            # Check what kind of message it is
            if data.get("type") == "query":
                user_query = data.get("content")
                session_id = data.get("session_id", "default")

                # We need to simulate streaming.
                # In a real app, you would modify the LangGraph agent to `yield` tokens
                # For this beginner tutorial, we will fetch the whole answer,
                # and then stream it out word-by-word to simulate the effect.

                response = await generate_chat_response(lease_id, user_query, session_id)
                answer_text = response["answer"]

                # SIMULATE STREAMING (Sending tokens one by one)
                words = answer_text.split(" ")
                for word in words:
                    # Send a chunk (token) to the frontend
                    await websocket.send_json({
                        "type": "token",
                        "content": word + " " # add the space back
                    })
                    # Pause for a tiny fraction of a second to make it look like typing
                    await asyncio.sleep(0.05)

                # Send the citations after the text is done
                await websocket.send_json({
                    "type": "citations",
                    "data": response["citations"]
                })
    except WebSocketDisconnect:
        pass

```

```

    })

    # Tell the frontend we are completely done with this question
    await websocket.send_json({
        "type": "done"
    })

except WebSocketDisconnect:
    # The user closed the browser tab ("Hung up the phone")
    print(f"Client disconnected from lease {lease_id}")

except Exception as e:
    # Something crashed! Let the frontend know.
    await websocket.send_json({
        "type": "error",
        "message": str(e)
    })

```

What just happened?

1. `await websocket.accept()` : Establishes the real-time connection.
2. `while True:` : A continuous loop that waits for the user to send a message.
3. `receive_text()` & `json.loads()` : Listens for the user's data. We expect it to look like `{"type": "query", "content": " ... "}`.
4. **Simulated Streaming**: We split the final answer into words and use a `for` loop combined with `asyncio.sleep(0.05)` to send them piece-by-piece via `send_json`.
5. **Exception Handling**: If the user closes the tab, a `WebSocketDisconnect` error is thrown, which we catch gracefully so our server doesn't crash.

● CAUTION

****Real Streaming vs Simulated****: In this tutorial, we simulate streaming by getting the whole answer first, then breaking it apart. This is to keep LangGraph simple. In production, you would configure the LlamaIndex LLM to use `stream=True` and pass those chunks directly into the WebSocket.

2. Route Registration

Now that we have our WebSocket route, we need to tell FastAPI about it. Create or edit `backend/app/api/routes/__init__.py` :


```
# backend/app/api/routes/__init__.py

from fastapi import APIRouter
from .chat import router as chat_router

# Create a master router
api_router = APIRouter()

# Attach our chat router to the master router
api_router.include_router(chat_router, tags=["chat"])

# You would also include your document upload routes here (from Part 1)
# from .documents import router as doc_router
# api_router.include_router(doc_router, tags=["documents"])
```

Make sure in your `backend/app/main.py` (which you made in Part 1), you include this master router:

```
# inside main.py
from app.api.routes import api_router

app.include_router(api_router)
```

Chapter 9: Putting the Backend Together

We have written a lot of code! Let's get it running on your Windows machine.

1. Install Dependencies

Open your Windows command prompt or PowerShell. Navigate to your backend folder and install the required Python packages.

```
cd C:\path\to\your\LeaseGPT\backend

# Ensure you are in your virtual environment if you made one!
# Then install dependencies.
pip install fastapi uvicorn langgraph llama-index
```

2. Start the Server

To run our FastAPI application, we use a server called **Uvicorn**.

```
# Run the app.main file, looking for the 'app' variable.
# --reload means it will automatically restart if you save a code change!
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

You should see green text saying `Application startup complete.` .

3. Test the API Manually (Swagger UI)

FastAPI gives you a free, auto-generated testing website!

1. Open your browser and go to: `http://localhost:8000/docs`
2. You will see a beautiful interface listing all your routes.
3. *Note: WebSockets don't easily test in Swagger.*

4. Test the WebSocket Real-Time Chat

To test a WebSocket on Windows without building a frontend React app yet, we can use a simple HTML file.

Create a temporary file on your Desktop called `test_chat.html` :

```

<!DOCTYPE html>
<html>
<body>
  <h2>LeaseGPT Chat Test</h2>
  <input type="text" id="query" placeholder="Ask a question ... " style="width: 300px;"
  <button onclick="sendMsg()">Send</button>
  <div id="chatbox" style="margin-top: 20px; border: 1px solid black; padding: 10px;

  <script>
    // Connect to our FastAPI WebSocket
    // Fake lease UUID for testing
    let ws = new WebSocket("ws://localhost:8000/ws/chat/123e4567-e89b-12d3-a456-4266
    let chatbox = document.getElementById("chatbox");

    ws.onmessage = function(event) {
      let data = JSON.parse(event.data);

      if (data.type === "token") {
        // Append the word as it arrives!
        chatbox.innerHTML += data.content;
      } else if (data.type === "citations") {
        chatbox.innerHTML += "<br><br><i>Source: Page " + data.data[0].page_num
      }
    };

    function sendMsg() {
      let input = document.getElementById("query");
      chatbox.innerHTML += "<b>You:</b> " + input.value + "<br><b>AI:</b> ";

      // Send the JSON format our backend expects
      ws.send(JSON.stringify({
        type: "query",
        content: input.value
      }));

      input.value = ""; // clear input
    }
  </script>
</body>
</html>

```

Double click `test_chat.html` to open it in Chrome or Edge.

Type a question like "What is the rent?" and hit Send.

You should see the AI's answer magically typing out onto the screen word by word!

Conclusion of Part 3

Congratulations! You have successfully built:

1. A **LangGraph Agent** that can analyze intent, retrieve context, judge relevance, and format answers.
2. A **WebSocket server** capable of pushing real-time tokens to a browser, giving that premium ChatGPT-style feel.

In **Part 4**, we will move to the Frontend and build the sleek Next.js React interface so users can drag-and-drop their PDFs and chat in a beautiful UI.

Chapter 10: Building the Frontend (Next.js + TypeScript + TailwindCSS)

Welcome to Part 4 of the LeaseGPT tutorial! By now, you have a fully functioning backend that can process PDFs, extract text, and answer questions using AI. But it's just a backend — it doesn't have a user interface (UI) yet.

In this chapter, we are going to build a **STUNNING, premium, dark-themed frontend** for LeaseGPT. It will look similar to modern AI chat interfaces like ChatGPT or Perplexity, complete with glassmorphism effects (that cool, frosted-glass look), smooth animations, and a seamless chat experience.

Before we write code, let's break down the technologies we'll be using. Think of building a website like building a house:

- **HTML** is the structure (walls, floors, roof).
- **CSS** is the design (paint, wallpaper, furniture).
- **JavaScript** is the electricity and plumbing (making things light up and flush).

For our modern app, we'll use tools that make building this house much faster and stronger:

1. **Next.js (The Blueprint & Foundation):** Next.js is a "framework" built on top of React. If plain React is like buying wood and nails to build a house, Next.js is like buying a prefabricated house where the plumbing, routing (moving between rooms/pages), and server features are already built-in. It uses the **App Router**, meaning we create pages just by making folders!
2. **TypeScript (The Building Inspector):** TypeScript is JavaScript with "types." Imagine you have moving boxes. In regular JavaScript, a box is just a box. You can put clothes in it, take them out, and put dishes in it. If you forget and throw the box, the dishes break. In TypeScript, you label the box "Clothes Only." If you try to put dishes in, TypeScript yells at you *before* you seal the box. It catches bugs before you run the code!
3. **TailwindCSS (The Instant Paint & Decorator):** Instead of writing separate CSS files (which can get messy), Tailwind is a "utility-first" CSS framework. You apply styles directly to your HTML elements using class names. For example, `className="bg-blue-500 text-white p-4 rounded-lg"` means "blue background, white text, padding of size 4, rounded corners." It's like having a catalog of pre-mixed paints you can just slap onto walls instantly.
4. **React Hooks (The Smart Appliances):** Hooks are special functions in React that let your components "hook into" React features. For example, `useState` lets a component remember things (like if a button was clicked), and `useEffect` lets a component do things

automatically (like fetching data when the page loads). **Custom Hooks** are hooks we write ourselves to bundle up complex logic so we can reuse it easily.

Let's dive in!

10.1: Initialize the Next.js Project

First, we need to create our frontend project. Open a new terminal window in your project's root folder.

⚠ IMPORTANT

Make sure you are in the main `leasegpt` folder (the parent folder of your backend) before running this command.

Run this exact command:

```
# Create a new Next.js app in a folder named 'frontend'
npx -y create-next-app@latest ./frontend --typescript --tailwind --eslint --app --src-dir
```

What just happened?

Let's break down that command:

- `npx create-next-app@latest` : This downloads and runs the Next.js setup wizard.
- `./frontend` : This tells it to create the app inside a folder called `frontend`.
- `--typescript` : We want to use TypeScript (our building inspector).
- `--tailwind` : We want TailwindCSS pre-configured.
- `--eslint` : Adds a tool that checks our code for formatting errors.
- `--app` : Uses the modern Next.js 14+ "App Router" (where folders define URL paths).
- `--src-dir` : Puts all our code inside a `src` folder (keeps things neat).
- `--import-alias "@/*"` : A shortcut! Instead of writing `../../../../components/Button`, we can just write `@/components/Button`.
- `--use-npm` : Uses npm as our package manager.

Once it finishes, open the `frontend` folder in your code editor. Your folder structure will look roughly like this:

```

frontend/
├── src/
│   ├── app/           # Where our pages live
│   │   ├── layout.tsx # The main wrapper for all pages
│   │   ├── page.tsx   # The home page (localhost:3000/)
│   │   └── globals.css # Global styles
│   ├── public/        # Static files like images or icons
│   ├── tailwind.config.ts # Tailwind customization
│   ├── package.json   # List of installed packages
│   └── tsconfig.json  # TypeScript settings

```

10.2: Configure TailwindCSS Theme

We want LeaseGPT to look *premium*. Let's set up a dark theme with glassmorphism and smooth animations.

First, let's install some helpful icons. We'll use `lucide-react`, a popular icon library.

```

cd frontend
npm install lucide-react clsx tailwind-merge framer-motion react-markdown remark-gfm

```

(We also installed `clsx` and `tailwind-merge` to help us combine Tailwind classes neatly, `framer-motion` for animations, and `react-markdown` to render the AI's responses beautifully.)

Open `tailwind.config.ts` and replace its entire contents with this:

```
// frontend/tailwind.config.ts
import type { Config } from "tailwindcss";

const config: Config = {
  // Tell Tailwind to look for classes in these files
  content: [
    "./src/pages/**/*.{js,ts,jsx,tsx,mdx}",
    "./src/components/**/*.{js,ts,jsx,tsx,mdx}",
    "./src/app/**/*.{js,ts,jsx,tsx,mdx}",
  ],
  theme: {
    extend: {
      // We are adding custom colors for our dark theme
      colors: {
        background: "#0f1115", // Deep dark slate background
        foreground: "#f8fafc", // Off-white text
        primary: {
          DEFAULT: "#6366f1", // Indigo accent
          hover: "#4f46e5",
        },
        surface: {
          DEFAULT: "rgba(30, 41, 59, 0.7)", // Semi-transparent slate for glass effect
          border: "rgba(148, 163, 184, 0.1)", // Very faint border
        }
      },
      // Custom animations!
      keyframes: {
        "fade-in": {
          "0%": { opacity: "0" },
          "100%": { opacity: "1" },
        },
        "slide-up": {
          "0%": { opacity: "0", transform: "translateY(20px)" },
          "100%": { opacity: "1", transform: "translateY(0)" },
        },
        "pulse-dot": {
          "0%, 100%": { opacity: "0.4", transform: "scale(0.8)" },
          "50%": { opacity: "1", transform: "scale(1.2)" },
        }
      },
      animation: {
        "fade-in": "fade-in 0.3s ease-out forwards",
        "slide-up": "slide-up 0.4s ease-out forwards",
        "pulse-dot": "pulse-dot 1.5s infinite ease-in-out",
      }
    },
  },
  plugins: [],
};

export default config;
```

Next, open `src/app/globals.css` and replace it entirely with:


```

/* frontend/src/app/globals.css */
@tailwind base;
@tailwind components;
@tailwind utilities;

:root {
  color-scheme: dark;
}

body {
  background-color: theme('colors.background');
  color: theme('colors.foreground');
  /* The Inter font looks very clean and modern */
  font-family: var(--font-inter), sans-serif;
}

/* Custom scrollbar for a polished look */
::-webkit-scrollbar {
  width: 8px;
  height: 8px;
}

::-webkit-scrollbar-track {
  background: transparent;
}

::-webkit-scrollbar-thumb {
  background: rgba(148, 163, 184, 0.2);
  border-radius: 10px;
}

::-webkit-scrollbar-thumb:hover {
  background: rgba(148, 163, 184, 0.4);
}

/* A utility class for our glassmorphism effect */
.glass-panel {
  background: theme('colors.surface.DEFAULT');
  backdrop-filter: blur(12px); /* This makes what's behind it blurry! */
  -webkit-backdrop-filter: blur(12px);
  border: 1px solid theme('colors.surface.border');
  box-shadow: 0 4px 30px rgba(0, 0, 0, 0.1);
}

```

What just happened?

We set up our design system! We told Tailwind to use a specific very dark background color (#0f1115), created some custom animations for sliding elements onto the screen, and created a `.glass-panel` CSS class that uses `backdrop-filter: blur()` to create a frosted glass effect that looks incredibly premium.

10.3: TypeScript Types

Before we build components, we need to define our data types. Remember, TypeScript is our "Building Inspector" making sure our data boxes are labeled correctly.

Create a new file `src/lib/types.ts` :

(Note: You might need to create the `lib` folder inside `src` first)

```
// frontend/src/lib/types.ts

/**
 * Represents a Lease document in our system.
 * This must match what our FastAPI backend returns!
 */
export interface Lease {
  id: string; // A unique ID, e.g. a UUID
  filename: string; // Original name, e.g., "apartment_lease.pdf"
  status: "processing" | "ready" | "error"; // Current state
  pageCount: number; // How many pages in the PDF
  createdAt: string; // ISO Date string
}

/**
 * Represents a snippet of text cited by the AI to prove its answer.
 */
export interface Citation {
  chunkId: string; // ID of the text chunk in our vector database
  pageNumber: number; // Which page of the lease it's on
  sectionTitle?: string; // Optional title, e.g., "7. Utilities"
  snippet: string; // The actual text from the lease
}

/**
 * Represents a single message in the chat window.
 */
export interface ChatMessage {
  id: string; // Unique ID for the message
  role: "user" | "assistant"; // Who sent it?
  content: string; // The text of the message
  citations?: Citation[]; // If assistant, it might have citations
  createdAt: string; // ISO Date string
}

/**
 * These are the different types of messages our WebSocket can send us.
 * This is called a "Discriminated Union" in TypeScript.
 * It means the object can be ONE of these shapes, determined by the 'type' field.
 */
export type WebSocketMessage =
  | { type: "token"; content: string } // A piece of the AI's answer
  | { type: "citations"; citations: Citation[] } // References
  | { type: "done" } // AI finished typing
  | { type: "error"; message: string }; // Something broke

/**
 * What the backend returns when we successfully upload a file
 */
export interface UploadResponse {
  leaseId: string;
  message: string;
}
```

What just happened?

We defined the shapes of our data using `interface` and `type` .

A **Discriminated Union** (like `WebSocketMessage`) is a fantastic TypeScript feature. It says: "If the `type` is 'token', then I guarantee this object has a `content` string. But if `type` is 'error', it will have a `message` string instead." This prevents us from accidentally trying to read `content` on an error message!

10.4: API Client

We need a way to talk to our FastAPI backend for standard HTTP requests (uploading files, getting lease status).

Create `src/lib/api.ts` :

```
// frontend/src/lib/api.ts
import { Lease, UploadResponse } from "../types";

// Get the backend URL from our environment variables, default to localhost:8000
const API_BASE_URL = process.env.NEXT_PUBLIC_API_URL || "http://localhost:8000/api";

/**
 * Uploads a lease file (PDF or Image) to the backend.
 * @param file The file selected by the user
 * @returns An UploadResponse containing the new leaseId
 */
export async function uploadLease(file: File): Promise<UploadResponse> {
  // 1. Create a FormData object (this is how you send files in JavaScript)
  const formData = new FormData();
  formData.append("file", file);

  // 2. Make a POST request to our FastAPI backend
  const response = await fetch(`${API_BASE_URL}/leases/upload`, {
    method: "POST",
    body: formData,
  });

  // 3. If the server returns an error (like a 500 or 400), throw an exception
  if (!response.ok) {
    const errorData = await response.json().catch(() => ({}));
    throw new Error(errorData.detail || "Failed to upload file");
  }

  // 4. Parse and return the JSON response
  return response.json();
}

/**
 * Fetches the current status and details of a specific lease.
 * Used to check if processing is done!
 */
export async function getLease(leaseId: string): Promise<Lease> {
  const response = await fetch(`${API_BASE_URL}/leases/${leaseId}`);

  if (!response.ok) {
    throw new Error("Failed to fetch lease data");
  }

  return response.json();
}

/**
 * Deletes a lease from the backend.
 */
export async function deleteLease(leaseId: string): Promise<void> {
  const response = await fetch(`${API_BASE_URL}/leases/${leaseId}`, {
    method: "DELETE",
  });

  if (!response.ok) {
    throw new Error("Failed to delete lease");
  }
}
```

```
}  
}
```

NOTE

We use the prefix ``NEXT_PUBLIC_`` for our environment variables so Next.js knows it is safe to include this variable in the frontend browser code. If a variable doesn't have this prefix, Next.js keeps it secret on the server!

10.5: WebSocket Hook

Now for the magic. We need to connect to our backend's WebSocket to receive the streaming AI responses (typing letter by letter).

React Hooks can be tricky with WebSockets because React components render multiple times, and we only want to connect *once*. We'll build a **Custom Hook** to manage this.

Create a folder `src/hooks` and a file `src/hooks/useWebSocket.ts` :

```
// frontend/src/hooks/useWebSocket.ts
import { useState, useEffect, useRef, useCallback } from "react";
import { WebSocketMessage } from "@lib/types";

// Get the WebSocket URL. Note we change http:// to ws://
const WS_BASE_URL = (process.env.NEXT_PUBLIC_API_URL || "http://localhost:8000")
  .replace(/^http/, "ws");

type ConnectionState = "connecting" | "connected" | "disconnected" | "error";

export function useWebSocket(leaseId: string | null) {
  // --- STATE ---
  // useState holds data that, when changed, causes the screen to re-draw.
  const [connectionState, setConnectionState] = useState<ConnectionState>("disconnected");

  // --- REFS ---
  // useRef holds data that DOES NOT cause a re-draw when changed.
  // It's like a secret pocket the component can keep notes in.
  const wsRef = useRef<WebSocket | null>(null);
  const reconnectTimeoutRef = useRef<NodeJS.Timeout | null>(null);
  const reconnectAttemptsRef = useRef(0);

  // We let the caller provide a function to handle incoming messages
  const messageHandlerRef = useRef<(msg: WebSocketMessage) => void>();

  /**
   * We use this so the component can update its handler function
   * without triggering the WebSocket to disconnect and reconnect.
   */
  const onMessage = useCallback((handler: (msg: WebSocketMessage) => void) => {
    messageHandlerRef.current = handler;
  }, []);

  /**
   * Sends a text query to the backend over the active WebSocket.
   */
  const sendMessage = useCallback((query: string) => {
    if (wsRef.current && wsRef.current.readyState === WebSocket.OPEN) {
      // The backend expects a simple JSON object: { "query": "my question" }
      wsRef.current.send(JSON.stringify({ query }));
    } else {
      console.error("WebSocket is not connected");
    }
  }, []);

  /**
   * The main connection logic. Wrapped in useCallback so we can call it recursively for
   */
  const connect = useCallback(() => {
    if (!leaseId) return;

    setConnectionState("connecting");

    // Create the connection to our FastAPI endpoint
    const ws = new WebSocket(`${WS_BASE_URL}/ws/chat/${leaseId}`);
    wsRef.current = ws;
  }, []);
}
```

```

ws.onopen = () => {
  console.log("WebSocket connected!");
  setConnectionState("connected");
  reconnectAttemptsRef.current = 0; // Reset attempts on success
};

ws.onmessage = (event) => {
  try {
    // Parse the incoming JSON message from the backend
    const data: WebSocketMessage = JSON.parse(event.data);

    // Pass it to the handler if one was provided
    if (messageHandlerRef.current) {
      messageHandlerRef.current(data);
    }
  } catch (e) {
    console.error("Failed to parse WebSocket message", e);
  }
};

ws.onclose = () => {
  setConnectionState("disconnected");

  // Exponential Backoff Reconnection strategy
  // 1s, 2s, 4s, 8s, up to max 30s
  const attempts = reconnectAttemptsRef.current;
  const backoffMs = Math.min(1000 * Math.pow(2, attempts), 30000);

  console.log(`WebSocket closed. Reconnecting in ${backoffMs}ms ...`);

  reconnectTimeoutRef.current = setTimeout(() => {
    reconnectAttemptsRef.current += 1;
    connect();
  }, backoffMs);
};

ws.onerror = () => {
  // We don't set error state here because onclose will fire immediately after
  console.error("WebSocket error occurred");
};
}, [leaseId]);

/**
 * useEffect runs automatically when the component mounts or when its dependencies (1
 */
useEffect(() => {
  connect();

  // The return function is the "Cleanup Function".
  // React runs this when the component unmounts or before running the effect again.
  return () => {
    if (reconnectTimeoutRef.current) clearTimeout(reconnectTimeoutRef.current);
    if (wsRef.current) {
      // Remove the onclose handler so it doesn't try to reconnect when we intentionally
      wsRef.current.onclose = null;
      wsRef.current.close();
    }
  };
};

```



```

    }, [connect]); // Only re-run if 'connect' changes (which only happens if leaseId cha

    return {
      connectionState,
      sendMessage,
      onMessage
    };
  }
}

```

What just happened?

We created a robust WebSocket hook!

- **useRef vs useState** : This is a core React concept. If you put the WebSocket inside `useState`, changing it would cause the component to redraw. We don't want the screen to redraw just because we saved a reference to the connection. So we put it in `useRef.current`.
- **Exponential Backoff**: If the connection drops, we don't spam the server. We wait 1 second, then 2, then 4, then 8. This is a professional-grade pattern for networking.
- **The Cleanup Function**: When you leave the chat page, React calls the `return () => { ... }` function in `useEffect`. This cleanly closes the WebSocket so we don't leak memory.

10.6: Chat Hook

Now let's build the hook that manages the list of messages in our chat window. It will use the WebSocket hook we just built.

Create `src/hooks/useChat.ts` :

```

// frontend/src/hooks/useChat.ts
import { useState, useCallback, useEffect } from "react";
import { ChatMessage, Citation } from "@lib/types";
import { useWebSocket } from "../useWebSocket";

export function useChat(leaseId: string | null) {
  // Store the list of messages in the chat
  const [messages, setMessages] = useState<ChatMessage[]>([]);
  // Are we currently receiving tokens from the AI?
  const [isStreaming, setIsStreaming] = useState(false);

  // Bring in our WebSocket hook
  const { connectionState, sendMessage, onMessage } = useWebSocket(leaseId);

  // Generate a random ID for new messages (UUID v4 format is best, but this is simple)
  const generateId = () => Math.random().toString(36).substring(2, 9);

  // Set up the listener for incoming WebSocket messages
  useEffect(() => {
    onMessage((msg) => {
      if (msg.type === "token") {
        setIsStreaming(true);
        // We use the "functional update" form of setMessages.
        // It gives us the *current* state (prev) so we can modify it safely.
        setMessages((prev) => {
          const lastMsg = prev[prev.length - 1];

          // If the last message isn't from the assistant, create a new one
          if (!lastMsg || lastMsg.role !== "assistant") {
            const newAssistantMsg: ChatMessage = {
              id: generateId(),
              role: "assistant",
              content: msg.content,
              createdAt: new Date().toISOString(),
            };
            return [...prev, newAssistantMsg];
          }

          // Otherwise, append the new token to the existing assistant message
          const updatedMsg = {
            ...lastMsg,
            content: lastMsg.content + msg.content,
          };

          // Replace the last message in the array
          return [...prev.slice(0, -1), updatedMsg];
        });
      }
      else if (msg.type === "citations") {
        // Attach citations to the current assistant message
        setMessages((prev) => {
          const lastMsg = prev[prev.length - 1];
          if (!lastMsg || lastMsg.role !== "assistant") return prev;

          const updatedMsg = {
            ...lastMsg,
            citations: msg.citations,
          };
        });
      }
    });
  }, [onMessage]);
}

```

```

    };
    return [ ...prev.slice(0, -1), updatedMsg];
  });
}
else if (msg.type === "done") {
  setIsStreaming(false);
}
else if (msg.type === "error") {
  setIsStreaming(false);
  // Add a system error message to the chat
  const errorMsg: ChatMessage = {
    id: generateId(),
    role: "assistant",
    content: `*Error:* ${msg.message}`,
    createdAt: new Date().toISOString(),
  };
  setMessages((prev) => [ ...prev, errorMsg]);
}
});
}, [onMessage]);

/**
 * Called when the user clicks "Send"
 */
const sendQuery = useCallback((query: string) => {
  // 1. Don't send empty messages
  if (!query.trim()) return;

  // 2. Add the user's message to the chat immediately
  const userMsg: ChatMessage = {
    id: generateId(),
    role: "user",
    content: query,
    createdAt: new Date().toISOString(),
  };
  setMessages((prev) => [ ...prev, userMsg]);

  // 3. Send it to the backend via WebSocket
  sendMessage(query);
}, [sendMessage]);

return {
  messages,
  isStreaming,
  connectionState,
  sendQuery,
};
}

```

What just happened?

This hook acts as the "Brain" for our Chat window.

When a `token` arrives, it checks: "Is the last message from the assistant?"

- If no, it creates a new assistant message with that first word.
- If yes, it glues the new word onto the end of the existing message.

Because `setMessages` updates React state, the screen redraws every time a token arrives, creating the typing effect!

10.7: Upload Hook

We need a hook to manage the state of our file upload and polling for readiness. Since vectorizing a PDF takes time, our backend returns `status: "processing"`. We need to keep checking back until it says `ready`.

Create `src/hooks/useLeaseUpload.ts` :

```
// frontend/src/hooks/useLeaseUpload.ts
import { useState, useCallback } from "react";
import { uploadLease, getLease } from "@lib/api";

type UploadStatus = "idle" | "uploading" | "processing" | "ready" | "error";

export function useLeaseUpload() {
  const [status, setStatus] = useState<UploadStatus>("idle");
  const [progress, setProgress] = useState(0); // 0 to 100
  const [errorMsg, setErrorMsg] = useState<string | null>(null);
  const [readyLeaseId, setReadyLeaseId] = useState<string | null>(null);

  /**
   * Helper function that polls the backend every 2 seconds
   * until the lease is 'ready' or 'error'.
   */
  const pollProcessingStatus = async (leaseId: string) => {
    setStatus("processing");
    // Faux progress just for UI flair (goes from 50 to 95 slowly)
    let fauxProgress = 50;

    const pollInterval = setInterval(async () => {
      try {
        const leaseData = await getLease(leaseId);

        if (leaseData.status === "ready") {
          clearInterval(pollInterval);
          setProgress(100);
          setStatus("ready");
          setReadyLeaseId(leaseId);
        } else if (leaseData.status === "error") {
          clearInterval(pollInterval);
          setStatus("error");
          setErrorMsg("Failed to process document.");
        } else {
          // Still processing, bump faux progress a bit
          if (fauxProgress < 95) {
            fauxProgress += 5;
            setProgress(fauxProgress);
          }
        }
      } catch (e) {
        clearInterval(pollInterval);
        setStatus("error");
        setErrorMsg("Lost connection while checking status.");
      }
    }, 2000); // Check every 2 seconds
  };

  /**
   * Main function called when user drops a file
   */
  const uploadFile = useCallback(async (file: File) => {
    setStatus("uploading");
    setProgress(20); // Initial jump
    setErrorMsg(null);
  });
}
```

```

    try {
      const response = await uploadLease(file);
      setProgress(50); // Upload complete, backend has it

      // Now start polling until it's indexed
      pollProcessingStatus(response.leaseId);

    } catch (e: any) {
      setStatus("error");
      setErrorMsg(e.message || "An error occurred during upload.");
    }
  }, []);

  return {
    uploadFile,
    status,
    progress,
    errorMsg,
    readyLeaseId,
  };
}

```

10.8: UI Components

Now for the fun part: Building the visual blocks of our app! We will build these inside `src/components`.

1. The Button Component

Create `src/components/ui/Button.tsx`. We'll use `clsx` and `tailwind-merge` to combine Tailwind classes cleanly.

```
// frontend/src/components/ui/Button.tsx
import React from "react";
import { Loader2 } from "lucide-react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

// A utility function to safely merge tailwind classes
export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

// Extending standard HTML Button props
interface ButtonProps extends React.ButtonHTMLAttributes<HTMLButtonElement> {
  variant?: "primary" | "secondary" | "ghost";
  isLoading?: boolean;
}

export function Button({
  children,
  variant = "primary",
  isLoading = false,
  className,
  disabled,
  ...props
}: ButtonProps) {

  // Base styles applied to all buttons
  const baseStyles = "inline-flex items-center justify-center rounded-lg font-medium tr

  // Styles specific to the variant chosen
  const variants = {
    primary: "bg-primary text-white hover:bg-primary-hover shadow-lg shadow-primary/20",
    secondary: "bg-surface border border-surface-border text-foreground hover:bg-white/",
    ghost: "bg-transparent text-gray-400 hover:text-white hover:bg-white/10 focus:ring-
  };

  return (
    <button
      className={cn(baseStyles, variants[variant], "px-4 py-2", className)}
      disabled={disabled || isLoading}
      { ... props}
    >
      { /* If loading, show a spinning icon */}
      {isLoading && <Loader2 className="mr-2 h-4 w-4 animate-spin" />}
      {children}
    </button>
  );
}
```

2. FileUpload Component

Create `src/components/FileUpload.tsx`. This handles dragging and dropping files.

```
// frontend/src/components/FileUpload.tsx
import React, { useCallback, useState } from "react";
import { FileUp, File as FileIcon, X } from "lucide-react";
import { cn, Button } from "../ui/Button";

interface FileUploadProps {
  onUpload: (file: File) => void;
  isLoading: boolean;
}

export function FileUpload({ onUpload, isLoading }: FileUploadProps) {
  const [isDragging, setIsDragging] = useState(false);
  const [selectedFile, setSelectedFile] = useState<File | null>(null);
  const [error, setError] = useState<string | null>(null);

  // When user drags file over the zone
  const onDragOver = useCallback((e: React.DragEvent) => {
    e.preventDefault(); // Required to allow dropping
    setIsDragging(true);
  }, []);

  const onDragLeave = useCallback((e: React.DragEvent) => {
    e.preventDefault();
    setIsDragging(false);
  }, []);

  // When user drops the file
  const onDrop = useCallback((e: React.DragEvent) => {
    e.preventDefault();
    setIsDragging(false);
    setError(null);

    const file = e.dataTransfer.files[0];
    validateAndSelectFile(file);
  }, []);

  // When user clicks and selects a file via file browser
  const onChange = useCallback((e: React.ChangeEvent<HTMLInputElement>) => {
    setError(null);
    if (e.target.files && e.target.files[0]) {
      validateAndSelectFile(e.target.files[0]);
    }
  }, []);

  const validateAndSelectFile = (file: File) => {
    // Check size (e.g., 20MB max)
    if (file.size > 20 * 1024 * 1024) {
      setError("File is too large. Max size is 20MB.");
      return;
    }
    // Check type (Very basic check)
    if (!file.type.includes("pdf") && !file.type.includes("image")) {
      setError("Only PDFs and Images are supported.");
      return;
    }
    setSelectedFile(file);
  };
}
```



```

const handleUploadClick = () => {
  if (selectedFile) {
    onUpload(selectedFile);
  }
};

return (
  <div className="w-full max-w-xl mx-auto">
    {!selectedFile ? (
      // The Dropzone Area
      <label
        className={cn(
          "glass-panel flex flex-col items-center justify-center w-full h-64 rounded-
            isDragging ? "border-primary bg-primary/5" : "border-surface-border hover:b
        )}
        onDragOver={onDragOver}
        onDragLeave={onDragLeave}
        onDrop={onDrop}
      >
        <div className="flex flex-col items-center justify-center pt-5 pb-6 text-cent
          <div className="p-4 bg-surface rounded-full mb-4 shadow-lg">
            <FileUp className="w-8 h-8 text-primary" />
          </div>
          <p className="mb-2 text-lg font-semibold">Click to upload or drag and drop<
          <p className="text-sm text-gray-400">PDF, JPG, PNG or TIFF (MAX. 20MB)</p>
        </div>
        <input
          type="file"
          className="hidden"
          accept=".pdf,image/*"
          onChange={onFileChange}
        />
      </label>
    ) : (
      // File Selected View
      <div className="glass-panel p-6 rounded-2xl animate-fade-in">
        <div className="flex items-center justify-between mb-6">
          <div className="flex items-center space-x-4">
            <div className="p-3 bg-primary/20 rounded-lg text-primary">
              <FileIcon className="w-6 h-6" />
            </div>
            <div>
              <p className="text-sm font-medium truncate max-w-[200px] sm:max-w-[300p
                {selectedFile.name}
              </p>
              <p className="text-xs text-gray-400">
                {(selectedFile.size / 1024 / 1024).toFixed(2)} MB
              </p>
            </div>
          </div>
          <button
            onClick={() => setSelectedFile(null)}
            className="p-2 text-gray-400 hover:text-white transition-colors"
            disabled={isLoading}
          >
            <X className="w-5 h-5" />
          </button>
        </div>
      </div>
    )
  )
);

```

```

    </div>

    <Button
      className="w-full"
      onClick={handleUploadClick}
      isLoading={isLoading}
    >
      {isLoading ? "Processing ... " : "Analyze Document"}
    </Button>
  </div>
)}

{error && (
  <div className="mt-4 p-3 bg-red-500/10 border border-red-500/20 text-red-400 rounded">
    {error}
  </div>
)}
</div>
);
}

```

3. ProcessingStatus Component

Create `src/components/ProcessingStatus.tsx` . Shows progress while the backend vectorizes the PDF.

```
// frontend/src/components/ProcessingStatus.tsx
import React from "react";
import { CheckCircle2, Loader2 } from "lucide-react";
import { cn } from "../ui/Button";

interface ProcessingStatusProps {
  status: "idle" | "uploading" | "processing" | "ready" | "error";
  progress: number;
}

export function ProcessingStatus({ status, progress }: ProcessingStatusProps) {
  // Define our steps based on current status
  const steps = [
    { label: "Uploading file", done: progress ≥ 50 },
    { label: "Extracting text & images", done: progress ≥ 75 },
    { label: "Generating embeddings", done: status === "ready" },
  ];

  return (
    <div className="glass-panel p-8 rounded-2xl w-full max-w-xl mx-auto animate-fade-in">
      <div className="text-center mb-8">
        <h3 className="text-xl font-bold mb-2">Processing Document</h3>
        <p className="text-gray-400 text-sm">Our AI is reading your lease. This usually
      </div>

      { /* Progress Bar */ }
      <div className="w-full bg-surface border border-surface-border rounded-full h-2">
        <div
          className="bg-primary h-2 rounded-full transition-all duration-500 ease-out"
          style={{ width: `${progress}%` }}
        ></div>
      </div>

      { /* Step List */ }
      <div className="space-y-4">
        {steps.map((step, index) => {
          // Determine state of this specific step
          const isCurrent = !step.done && (index === 0 || steps[index - 1].done);

          return (
            <div key={index} className="flex items-center space-x-3">
              {step.done ? (
                <CheckCircle2 className="w-5 h-5 text-green-400" />
              ) : isCurrent && status === "error" ? (
                <Loader2 className="w-5 h-5 text-primary animate-spin" />
              ) : (
                <div className="w-5 h-5 rounded-full border-2 border-surface-border" />
              )}

              <span className={cn(
                "text-sm font-medium transition-colors duration-300",
                step.done ? "text-gray-300" : isCurrent ? "text-white" : "text-gray-500"
              )}>
                {step.label}
              </span>
            </div>
          );
        })}
      </div>
    </div>
  );
}
```

```
    }  
  }  
  </div>  
</div>  
)  
}
```

4. MessageBubble Component

Create `src/components/MessageBubble.tsx`. This renders individual chat messages and supports Markdown!

```
// frontend/src/components/MessageBubble.tsx
import React from "react";
import ReactMarkdown from "react-markdown";
import remarkGfm from "remark-gfm"; // Adds support for tables, task lists, etc.
import { User, Sparkles } from "lucide-react";
import { ChatMessage } from "@lib/types";
import { cn } from "../ui/Button";

interface MessageBubbleProps {
  message: ChatMessage;
  isStreaming?: boolean;
}

export function MessageBubble({ message, isStreaming }: MessageBubbleProps) {
  const isUser = message.role === "user";

  return (
    <div className={cn(
      "flex w-full mb-6 animate-slide-up",
      isUser ? "justify-end" : "justify-start"
    )}>
      <div className={cn(
        "flex max-w-[85%] sm:max-w-[75%]",
        isUser ? "flex-row-reverse" : "flex-row"
      )}>

        {/* Avatar Icon */}
        <div className={cn(
          "flex-shrink-0 w-8 h-8 rounded-full flex items-center justify-center mt-1",
          isUser ? "ml-3 bg-primary" : "mr-3 bg-surface border border-surface-border",
        )}>
          {isUser ? <User className="w-5 h-5 text-white" /> : <Sparkles className="w-5 h-5 text-white" />}
        </div>

        {/* Message Content Bubble */}
        <div className={cn(
          "px-5 py-4 rounded-2xl",
          isUser ? "bg-primary text-white rounded-tr-sm" : "glass-panel rounded-tl-sm"
        )}>

          {/* Render Markdown! prose classes style the HTML output of ReactMarkdown */}
          <div className={cn(
            "prose prose-sm max-w-none leading-relaxed",
            isUser ? "prose-invert text-white" : "prose-invert"
          )}>
            <ReactMarkdown remarkPlugins={[remarkGfm]}>
              {message.content}
            </ReactMarkdown>

            {/* Animated Typing Indicator */}
            {isStreaming && (
              <span className="inline-flex space-x-1 ml-1">
                <span className="w-1.5 h-1.5 bg-gray-400 rounded-full animate-pulse-dot">
                  <span className="w-1.5 h-1.5 bg-gray-400 rounded-full animate-pulse-dot">
                    <span className="w-1.5 h-1.5 bg-gray-400 rounded-full animate-pulse-dot">
                </span>
              </span>
            )}
          </div>
        </div>
      </div>
    </div>
  );
}
```

```

    </div>

    { /* Citations block (if the assistant provided them) */}
    {message.citations && message.citations.length > 0 && (
      <div className="mt-4 pt-3 border-t border-white/10">
        <p className="text-xs text-gray-400 mb-2 font-medium uppercase tracking-widely">
          Citations
        </p>
        <div className="flex flex-wrap gap-2">
          {message.citations.map((cite, idx) => (
            <div key={idx} className="group relative">
              { /* The Badge */}
              <span className="inline-flex items-center px-2 py-1 rounded bg-white text-gray-500 font-size-sm">
                Page {cite.pageNumber}
              </span>

              { /* Tooltip on hover */}
              <div className="absolute bottom-full left-0 mb-2 w-64 p-3 bg-gray-900 text-white font-size-sm">
                {cite.sectionTitle && <p className="font-bold text-white mb-1">{cite.sectionTitle}</p>
                  <p className="text-gray-300 italic line-clamp-4">{cite.snippet}</p>
                </div>
              </div>
            </div>
          ))}
        </div>
      </div>
    )}
  </div>
</div>
);
}

```

5. ChatInput Component

Create `src/components/ChatInput.tsx`. An auto-expanding textarea for typing queries.

```
// frontend/src/components/ChatInput.tsx
import React, { useState, useRef, useEffect } from "react";
import { SendHorizontal } from "lucide-react";
import { cn } from "../ui/Button";

interface ChatInputProps {
  onSend: (message: string) => void;
  disabled?: boolean;
}

export function ChatInput({ onSend, disabled }: ChatInputProps) {
  const [text, setText] = useState("");
  const textareaRef = useRef<HTMLTextAreaElement>(null);

  // Auto-resize the textarea based on content
  useEffect(() => {
    if (textareaRef.current) {
      textareaRef.current.style.height = "inherit";
      const computed = window.getComputedStyle(textareaRef.current);
      const height = textareaRef.current.scrollHeight;
      // Cap height at roughly 4 lines (96px)
      textareaRef.current.style.height = `${Math.min(height, 96)}px`;
    }
  }, [text]);

  const handleSend = () => {
    if (text.trim() && !disabled) {
      onSend(text.trim());
      setText("");
    }
  };

  const handleKeyDown = (e: React.KeyboardEvent) => {
    // Send on Enter, unless Shift is held down (which makes a new line)
    if (e.key === "Enter" && !e.shiftKey) {
      e.preventDefault();
      handleSend();
    }
  };

  return (
    <div className="glass-panel rounded-2xl p-2 flex items-end relative border-gray-700">
      <textarea
        ref={textareaRef}
        value={text}
        onChange={(e) => setText(e.target.value)}
        onKeyDown={handleKeyDown}
        placeholder={disabled ? "Please wait ..." : "Ask about your lease ..."}
        disabled={disabled}
        className="w-full max-h-32 bg-transparent text-white placeholder-gray-500 resize-none"
        rows={1}
      />
      <button
        onClick={handleSend}
        disabled={!text.trim() || disabled}
        className={cn(
          "mb-1 mr-1 p-2 rounded-xl transition-all duration-200 flex-shrink-0 flex item"
        )}
      >

```

```

        text.trim() && !disabled
        ? "bg-primary text-white shadow-lg hover:bg-primary-hover"
        : "bg-surface text-gray-500 cursor-not-allowed"
    }}
  >
    <SendHorizontal className="w-5 h-5" />
  </button>
</div>
);
}

```

10.9: Pages (Putting it all together)

Next.js uses a folder-based routing system. The `src/app/page.tsx` file is the homepage (/), and we'll create `src/app/chat/[leaseId]/page.tsx` for the chat page.

1. The Home Page (Upload)

Overwrite `src/app/page.tsx` :


```
// frontend/src/app/page.tsx
"use client"; // Tells Next.js this component uses browser features (like React Hooks)

import { useEffect } from "react";
import { useRouter } from "next/navigation";
import { Sparkles, FileText, Search } from "lucide-react";
import { FileUpload } from "@/components/FileUpload";
import { ProcessingStatus } from "@/components/ProcessingStatus";
import { useLeaseUpload } from "@/hooks/useLeaseUpload";

export default function Home() {
  const router = useRouter(); // For navigating to a new URL
  const { uploadFile, status, progress, errorMsg, readyLeaseId } = useLeaseUpload();

  // Watch for when the lease is ready, then redirect to the chat page!
  useEffect(() => {
    if (status === "ready" && readyLeaseId) {
      // Small delay just so the user sees the 100% complete state
      setTimeout(() => {
        router.push(`/chat/${readyLeaseId}`);
      }, 1000);
    }
  }, [status, readyLeaseId, router]);

  return (
    <main className="min-h-screen flex flex-col items-center justify-center p-6 relative">

      {/* Cool background glow effects */}
      <div className="absolute top-1/4 left-1/4 w-96 h-96 bg-primary/20 rounded-full bl-0 br-0"></div>
      <div className="absolute bottom-1/4 right-1/4 w-96 h-96 bg-purple-500/10 rounded-full bl-0 br-0"></div>

      <div className="w-full max-w-4xl mx-auto text-center mb-12 animate-slide-up">
        <div className="inline-flex items-center justify-center p-2 bg-primary/10 rounded">
          <Sparkles className="w-6 h-6 text-primary" />
        </div>
        <h1 className="text-5xl md:text-6xl font-extrabold mb-6 tracking-tight">
          Understand your lease in <span className="text-transparent bg-clip-text bg-gradient-to-r from-purple-500 to-blue-500">plain English</span>
        </h1>
        <p className="text-xl text-gray-400 max-w-2xl mx-auto">
          Upload your residential lease agreement and ask questions in plain English. No legalese.
        </p>
      </div>

      <div className="w-full max-w-3xl mx-auto animate-fade-in" style={{ animationDelay: 0.2s }}>
        {/* If idle or error, show upload box. If processing, show progress. */}
        {status === "idle" || status === "error" ? (
          <FileUpload onUpload={uploadFile} isLoading={status === "uploading"} />
        ) : (
          <ProcessingStatus status={status} progress={progress} />
        )}

        {status === "error" && errorMsg && (
          <p className="text-red-400 text-center mt-4">{errorMsg}</p>
        )}
      </div>

      {/* Feature blurbs */}
    </main>
  );
}
```

```

<div className="grid grid-cols-1 md:grid-cols-3 gap-8 mt-24 max-w-4xl mx-auto op
  <div className="flex flex-col items-center text-center">
    <FileText className="w-8 h-8 text-gray-400 mb-3" />
    <h3 className="font-semibold mb-1">Instant Analysis</h3>
    <p className="text-sm text-gray-500">We extract every clause and condition se
  </div>
  <div className="flex flex-col items-center text-center">
    <Search className="w-8 h-8 text-gray-400 mb-3" />
    <h3 className="font-semibold mb-1">Smart Citations</h3>
    <p className="text-sm text-gray-500">Answers include exact page numbers and c
  </div>
  <div className="flex flex-col items-center text-center">
    <Sparkles className="w-8 h-8 text-gray-400 mb-3" />
    <h3 className="font-semibold mb-1">AI Powered</h3>
    <p className="text-sm text-gray-500">Powered by advanced RAG and Large Language
  </div>
</div>
</main>
);
}

```

2. The Chat Page

Create the folders `src/app/chat/[leaseId]` and then create `src/app/chat/[leaseId]/page.tsx`.

NOTE

The brackets `[leaseId]` in the folder name is Next.js's way of creating a **Dynamic Route**. If you go to `/chat/123-abc`, Next.js will pass `123-abc` to the component as a parameter named `leaseId`.

```
// frontend/src/app/chat/[leaseId]/page.tsx
"use client";

import { useEffect, useRef, useState } from "react";
import { useParams, useRouter } from "next/navigation";
import { ArrowLeft, FileText, AlertCircle } from "lucide-react";
import { useChat } from "@hooks/useChat";
import { getLease } from "@lib/api";
import { Lease } from "@lib/types";
import { MessageBubble } from "@components/MessageBubble";
import { ChatInput } from "@components/ChatInput";
import { Button } from "@components/ui/Button";

export default function ChatPage() {
  const params = useParams(); // Gets variables from the URL
  const router = useRouter();
  const leaseId = params.leaseId as string;

  const [lease, setLease] = useState<Lease | null>(null);
  const [error, setError] = useState<string | null>(null);

  // Initialize our Chat hook!
  const { messages, isStreaming, sendQuery, connectionState } = useChat(leaseId);

  // A reference to the bottom of the message list so we can auto-scroll
  const messagesEndRef = useRef<HTMLDivElement>(null);

  // 1. Fetch lease details on load to get the filename
  useEffect(() => {
    async function loadLease() {
      try {
        const data = await getLease(leaseId);
        if (data.status !== "ready") {
          setError("This document is not ready yet.");
        } else {
          setLease(data);
        }
      } catch (e) {
        setError("Document not found.");
      }
    }
    loadLease();
  }, [leaseId]);

  // 2. Auto-scroll to bottom when messages change
  useEffect(() => {
    messagesEndRef.current?.scrollIntoView({ behavior: "smooth" });
  }, [messages, isStreaming]);

  if (error) {
    return (
      <div className="min-h-screen flex items-center justify-center">
        <div className="glass-panel p-8 rounded-2xl text-center">
          <AlertCircle className="w-12 h-12 text-red-400 mx-auto mb-4" />
          <h2 className="text-xl font-bold mb-2">Error</h2>
          <p className="text-gray-400 mb-6">{error}</p>
          <Button onClick={() => router.push("/")}>Go Home</Button>
        </div>
      </div>
    );
  }
}
```



```

        </div>
      </div>
    ) : (
      // Message List
      <div className="pb-4">
        {messages.map((msg, index) => (
          <MessageBubble
            key={msg.id}
            message={msg}
            // Pass true to isStreaming ONLY for the very last message, IF we are
            isStreaming={isStreaming && index === messages.length - 1 && msg.role === "assistant"}
          />
        ))}
        {/* Invisible div to scroll to */}
        <div ref={messagesEndRef} />
      </div>
    )}
  </div>
</div>

{/* Input Area */}
<div className="p-4 sm:p-6 bg-gradient-to-t from-background via-background to-transparent">
  <div className="max-w-3xl mx-auto">
    <ChatInput
      onSend={sendQuery}
      disabled={isStreaming || connectionState !== "connected"}
    />
    <p className="text-center text-xs text-gray-500 mt-3">
      AI can make mistakes. Always verify important legal details in the original
    </p>
  </div>
</div>

</div>
);
}

```

10.10: Running the Frontend

You're done coding! Let's fire it up.

1. Ensure your backend is running in a separate terminal:

```
powershell cd leasegpt/backend poetry run uvicorn main:app --reload
```

2. Open a new terminal for the frontend.

3. Make sure you are in `leasegpt/frontend`.

4. Install all the dependencies you added:

```
powershell npm install
```

5. (Optional) Create a `.env.local` file in the `frontend` folder if your backend isn't on `localhost:8000`. By default, our code falls back to `localhost:8000`, so you can skip this if you're running locally.

6. Start the Next.js development server:

```
powershell npm run dev
```

7. Open your browser and go to `http://localhost:3000`.

You should see your stunning new dark-mode interface!

Try dragging a PDF lease into the drop zone. Watch the progress bar go as the backend processes it. When it finishes, you'll be whisked away to the chat page where you can start asking questions. As the AI replies, you'll see the words stream in smoothly, just like ChatGPT!

Congratulations! You have built a complete full-stack AI application using FastAPI, LangGraph, vector databases, and Next.js!

LeaseGPT Tutorial - Part 5: Cloud Deployment, DevOps, and Testing

Welcome to the grand finale of the LeaseGPT tutorial! 🚀

In Parts 1-4, we built an incredible system locally. We can upload a lease, extract its text, chunk it, generate embeddings, and chat with it using an AI agent.

But right now, it only lives on your computer. If you close your laptop, LeaseGPT goes to sleep. In this part, we are going to make it ready for the real world. We will learn how to process files in the cloud (AWS), package our app so it runs anywhere (Docker), orchestrate multiple copies of it (Kubernetes), automatically test and deploy our code (CI/CD), and ensure our AI isn't making things up (Testing & Evaluation).

Even if you've never touched a server before, don't panic! We are going to explain everything from scratch using simple analogies. Let's dive in!

Chapter 11: AWS Serverless Pipeline (Lambda + Step Functions)

When a user uploads a 100-page PDF, extracting text and generating embeddings takes time. If we do this in our main web server, the user's browser will freeze while it waits. Instead, we want to hand this heavy lifting off to a background worker in the cloud.

We're going to use **AWS Serverless** for this.

Core Concepts Explained for Beginners

- **What is AWS Lambda?** Imagine you want a specialized meal. You could build a full kitchen, buy ingredients, and hire a chef to stand there 24/7 (this is like a traditional server). Or, you could just use a food delivery app: you order exactly when you want it, someone else's kitchen makes it, and you only pay for that one meal. **Lambda** is the food delivery of code. You write a function, upload it to AWS, and AWS automatically spins up a "kitchen" to run your code *only* when an event happens (like a file being uploaded). You pay zero when it's not running.
- **What is AWS Step Functions?** Imagine a complex recipe. Step 1: Chop veggies. Step 2: If veggies are carrots, boil them; if they are onions, fry them. Step 3: Serve. **Step Functions** is a

visual workflow orchestrator that links multiple Lambda functions together like steps in a recipe. It handles passing data from one step to the next and catching errors (like if the pan catches fire).

- **What is a Docker-based Lambda?** Normally, you zip your Python code and upload it to Lambda. But if your code needs large external libraries (like PyMuPDF for PDFs), the zip gets too big and messy. By packaging our Lambda as a **Docker container**, we neatly box up our code and all its heavy dependencies into one standard package.
- **What is IAM (Identity and Access Management)?** IAM is like a bouncer at an exclusive club. It checks IDs and permissions. By default, an AWS Lambda function has no permission to read your database or access your files. You have to give the Lambda an "IAM Role" (a VIP pass) that explicitly says, "You are allowed to read from this specific bucket, and nothing else."
- **What is LocalStack?** AWS costs money and requires an internet connection. **LocalStack** is a brilliant tool that creates a "fake" miniature AWS right on your laptop. You can test Lambdas and S3 buckets locally for free before deploying to the real AWS.

✓ TIP

****The AWS Free Tier**** is incredibly generous for beginners: - ****Lambda:**** 1 million free requests per month, and 400,000 GB-seconds of compute time per month. This is ****ALWAYS free****, not just for the first year! - ****Step Functions:**** 4,000 state transitions per month (****ALWAYS free****). - ****S3 (Simple Storage Service):**** 5GB of storage, 20,000 GET requests, and 2,000 PUT requests per month (Free for the first 12 months). - ****Textract:**** 1,000 pages per month for DetectDocumentText (Free for the first 3 months).

11.1 The Classification Lambda

The first step in our pipeline is figuring out what kind of file the user uploaded. Is it a clean text PDF, a scanned PDF (needs OCR), or an image?

Create `lambdas/classify/handler.py` :


```
# lambdas/classify/handler.py

import json                # For parsing JSON data
import urllib.parse        # For decoding URL-encoded file names
import boto3              # The official AWS SDK for Python - lets us talk to AWS services
import fitz               # PyMuPDF library for reading PDF files
import os                 # For reading environment variables

# Create a client to talk to S3 (Amazon's file storage)
# We use endpoint_url so it can talk to LocalStack when testing locally
s3 = boto3.client('s3', endpoint_url=os.getenv('AWS_ENDPOINT_URL'))

def handler(event, context):
    """
    This is the entry point! AWS Lambda calls this function when triggered.

    Arguments:
    - event: A dictionary containing the data about what triggered the Lambda (e.g., the event structure is deeply nested JSON provided by AWS)
    - context: Information about the environment the Lambda is running in (we usually ignore this)
    """
    print("Classify Lambda started!")

    # 1. Extract the bucket name and file name (key) from the incoming event
    # The event structure is deeply nested JSON provided by AWS
    bucket = event['detail']['bucket']['name']

    # S3 replaces spaces in filenames with '+', so we unquote it to get the real name
    key = urllib.parse.unquote_plus(event['detail']['object']['key'])

    # 2. Download the file from S3 to a temporary folder inside the Lambda
    # Lambdas only let you write files to the '/tmp' directory!
    download_path = f'/tmp/{os.path.basename(key)}'
    print(f"Downloading {key} from bucket {bucket} to {download_path}")

    s3.download_file(bucket, key, download_path)

    # 3. Determine the file type
    file_type = "unknown"

    # Check the file extension
    if key.lower().endswith(('.png', '.jpg', '.jpeg')):
        file_type = 'image'
    elif key.lower().endswith('.pdf'):
        # If it's a PDF, we need to open it to see if it has selectable text
        doc = fitz.open(download_path)
        has_text = False

        # Check the first few pages
        for page_num in range(min(3, len(doc))):
            page = doc.load_page(page_num)
            text = page.get_text()
            # If we find a decent chunk of text, it's a Text PDF
            if len(text.strip()) > 50:
                has_text = True
                break

    if has_text:
```

```

        file_type = 'text_pdf'
    else:
        file_type = 'scanned_pdf' # No text found, needs OCR!

    doc.close()

# 4. Clean up the temporary file so we don't run out of space on future runs
os.remove(download_path)

# 5. Return the result. This JSON will be passed to the NEXT step in Step Functions
return {
    'statusCode': 200,
    'bucket': bucket,
    'key': key,
    'file_type': file_type
}

```

What just happened?

1. We defined a `handler(event, context)` function. This is the exact signature AWS expects.
2. We grabbed the file details from the `event` dictionary.
3. We downloaded the file to `/tmp/` (the only writable folder in a Lambda).
4. We used PyMuPDF (`fitz`) to peek inside the PDF. If it has text, we mark it `text_pdf` . Otherwise, it's an image or a `scanned_pdf` .
5. We returned a dictionary. Step Functions will capture this and use it to decide what to do next.

Now, let's containerize it. Create `lambdas/classify/Dockerfile` :

```

# lambdas/classify/Dockerfile

# 1. Start from Amazon's official base image for Python 3.12 Lambdas
# This image already has the Lambda runtime engine built in!
FROM public.ecr.aws/lambda/python:3.12

# 2. Copy our list of dependencies into the container
COPY requirements.txt ${LAMBDA_TASK_ROOT}

# 3. Install the dependencies using pip
# We install them directly into the Lambda task root directory
RUN pip install -r requirements.txt

# 4. Copy our actual Python code into the container
COPY handler.py ${LAMBDA_TASK_ROOT}

# 5. Tell the Lambda runtime where to find our function
# Format: filename.function_name
CMD [ "handler.handler" ]

```

And `lambdas/classify/requirements.txt` :

```
boto3==1.34.84  
PyMuPDF==1.24.1
```

11.2 The Extraction Lambda

Next, we extract the text. If it's a `text_pdf`, we use Python. If it's a `scanned_pdf`, we ask AWS Textract (an OCR service) to read the text from the images.

Create `lambdas/extract/handler.py` :

```

# lambdas/extract/handler.py

import json
import boto3
import fitz
import os
import time
import psycopg2 # For connecting to our PostgreSQL database

s3 = boto3.client('s3', endpoint_url=os.getenv('AWS_ENDPOINT_URL'))
textract = boto3.client('textract', endpoint_url=os.getenv('AWS_ENDPOINT_URL'))

def get_db_connection():
    # Connect to our PostgreSQL database using environment variables
    return psycopg2.connect(
        dbname=os.getenv("DB_NAME", "leasegpt"),
        user=os.getenv("DB_USER", "postgres"),
        password=os.getenv("DB_PASSWORD", "postgres"),
        host=os.getenv("DB_HOST", "host.docker.internal"),
        port=os.getenv("DB_PORT", "5432")
    )

def extract_text_from_pdf(file_path):
    # Standard PyMuPDF text extraction
    doc = fitz.open(file_path)
    full_text = ""
    for page in doc:
        full_text += page.get_text() + "\n\n"
    return full_text, len(doc)

def extract_text_with_textract(bucket, key):
    # AWS Textract is asynchronous. We say "start reading this", and it gives us a JobId
    response = textract.start_document_text_detection(
        DocumentLocation={'S3Object': {'Bucket': bucket, 'Name': key}}
    )
    job_id = response['JobId']

    # We must poll (check back repeatedly) until the job is done
    while True:
        status_response = textract.get_document_text_detection(JobId=job_id)
        status = status_response['JobStatus']
        if status == 'SUCCEEDED':
            break
        elif status == 'FAILED':
            raise Exception("Textract failed to process document")

        # Wait 2 seconds before checking again (Backoff)
        print("Waiting for Textract to finish...")
        time.sleep(2)

    # Once successful, stitch all the text blocks together
    extracted_text = ""
    for item in status_response['Blocks']:
        if item['BlockType'] == 'LINE':
            extracted_text += item['Text'] + "\n"

    return extracted_text, status_response['DocumentMetadata']['Pages']

```

```

def handler(event, context):
    print("Extract Lambda started!")

    # Get inputs passed from the Classify Lambda
    bucket = event['bucket']
    key = event['key']
    file_type = event['file_type']

    extracted_text = ""
    page_count = 0

    if file_type == 'text_pdf':
        download_path = f'/tmp/{os.path.basename(key)}'
        s3.download_file(bucket, key, download_path)
        extracted_text, page_count = extract_text_from_pdf(download_path)
        os.remove(download_path)

    elif file_type in ['scanned_pdf', 'image']:
        extracted_text, page_count = extract_text_with_textract(bucket, key)

    # We use the S3 key (filename) as our Lease ID in the database
    lease_id = os.path.basename(key).split('.')[0]

    # Save the extracted text back to our PostgreSQL database!
    conn = get_db_connection()
    cur = conn.cursor()

    # We use UPSERT (ON CONFLICT DO UPDATE) to insert or update the text
    cur.execute("""
        INSERT INTO leases (id, filename, extracted_text, page_count, status)
        VALUES (%s, %s, %s, %s, %s)
        ON CONFLICT (id) DO UPDATE
        SET extracted_text = EXCLUDED.extracted_text,
            page_count = EXCLUDED.page_count,
            status = EXCLUDED.status;
    """, (lease_id, os.path.basename(key), extracted_text, page_count, 'extracted'))

    conn.commit()
    cur.close()
    conn.close()

    return {
        'lease_id': lease_id,
        'bucket': bucket,
        'key': key,
        'status': 'extracted'
    }

```

What just happened?

1. We checked the `file_type` passed down from the classification step.
2. If it's a `text_pdf`, we download it and use Python to extract text.
3. If it's a `scanned_pdf`, we **don't** download it. Instead, we tell AWS Textract to read it straight from S3. Textract takes time, so we loop and `time.sleep(2)` until it finishes.

4. We take the final massive string of text and save it directly into our PostgreSQL `leases` table using the `psycopg2` library. We mark the database row status as `extracted`.

Create `lambdas/extract/Dockerfile` :

```
# lambdas/extract/Dockerfile
FROM public.ecr.aws/lambda/python:3.12
COPY requirements.txt ${LAMBDA_TASK_ROOT}
# Install dependencies and PostgreSQL development headers needed by psycopg2
RUN yum install -y postgresql-devel gcc && \
    pip install -r requirements.txt
COPY handler.py ${LAMBDA_TASK_ROOT}
CMD [ "handler.handler" ]
```

And `lambdas/extract/requirements.txt` :

```
boto3==1.34.84
PyMuPDF==1.24.1
psycopg2-binary==2.9.9
```

◆ WARNING

Compiling `psycopg2` inside a Docker container requires C compilers (`gcc`) and postgres headers. That's why we added `yum install -y postgresql-devel gcc` to the Dockerfile. Using Docker makes compiling these native C-extensions a breeze!

11.3 The Chunk & Embed Lambda

The final step: take the huge text, chop it into small overlapping chunks, turn them into vector embeddings, and save them to `pgvector`.

Create `lambdas/chunk_embed/handler.py` :

```
# lambdas/chunk_embed/handler.py

import os
import requests
import psycopg2
from llama_index.core.node_parser import SentenceSplitter

def get_db_connection():
    return psycopg2.connect(
        dbname=os.getenv("DB_NAME", "leasegpt"),
        user=os.getenv("DB_USER", "postgres"),
        password=os.getenv("DB_PASSWORD", "postgres"),
        host=os.getenv("DB_HOST", "host.docker.internal"),
        port=os.getenv("DB_PORT", "5432")
    )

def get_embedding(text):
    """
    Calls our local Ollama container to get embeddings.
    In production on AWS, this would call Amazon Bedrock or an OpenAI API.
    """
    ollama_host = os.getenv("OLLAMA_HOST", "http://host.docker.internal:11434")
    response = requests.post(f"{ollama_host}/api/embeddings", json={
        "model": "nomic-embed-text",
        "prompt": text
    })
    return response.json()['embedding']

def handler(event, context):
    print("Chunk & Embed Lambda started!")
    lease_id = event['lease_id']

    # 1. Fetch the extracted text from the database
    conn = get_db_connection()
    cur = conn.cursor()
    cur.execute("SELECT extracted_text FROM leases WHERE id = %s", (lease_id,))
    result = cur.fetchone()

    if not result:
        raise Exception(f"Lease {lease_id} not found in database!")

    full_text = result[0]

    # 2. Chunk the text using LlamaIndex
    # We split into 512-word chunks, overlapping by 50 words to maintain context
    splitter = SentenceSplitter(chunk_size=512, chunk_overlap=50)
    chunks = splitter.split_text(full_text)

    # 3. Get embeddings and save to pgvector
    for i, chunk_text in enumerate(chunks):
        print(f"Generating embedding for chunk {i+1}/{len(chunks)}")
        embedding = get_embedding(chunk_text)

        # Save chunk and vector array to database
        cur.execute("""
            INSERT INTO document_chunks (lease_id, chunk_index, text_content, embedding)
            VALUES (%s, %s, %s, %s::vector)
        """)
```

```

        "", (lease_id, i, chunk_text, embedding))

# 4. Mark the lease as fully processed!
cur.execute("UPDATE leases SET status = 'ready' WHERE id = %s", (lease_id,))
conn.commit()
cur.close()
conn.close()

return {
    'status': 'success',
    'lease_id': lease_id,
    'chunks_created': len(chunks)
}

```

Create `lambdas/chunk_embed/Dockerfile` :

```

# lambdas/chunk_embed/Dockerfile
FROM public.ecr.aws/lambda/python:3.12
COPY requirements.txt ${LAMBDA_TASK_ROOT}
RUN yum install -y postgresql-devel gcc && \
    pip install -r requirements.txt
COPY handler.py ${LAMBDA_TASK_ROOT}
CMD [ "handler.handler" ]

```

And `lambdas/chunk_embed/requirements.txt` :

```

psycopg2-binary==2.9.9
llama-index-core==0.10.30
requests==2.31.0

```

11.4 AWS Step Functions: The Orchestrator

Now we need to glue these Lambdas together. We define the workflow using **Amazon States Language (ASL)**, which is just a JSON file.

Create `infra/aws/step-functions.json` :


```
{
  "Comment": "LeaseGPT Document Processing Pipeline",
  "StartAt": "ClassifyDocument",
  "States": {
    "ClassifyDocument": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:us-east-1:000000000000:function:classify-lambda",
      "Next": "ProcessDocument",
      "Catch": [
        {
          "ErrorEquals": ["States.ALL"],
          "Next": "FailState"
        }
      ]
    },
    "ProcessDocument": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:us-east-1:000000000000:function:extract-lambda",
      "Next": "ChunkAndEmbed",
      "Catch": [
        {
          "ErrorEquals": ["States.ALL"],
          "Next": "FailState"
        }
      ]
    },
    "ChunkAndEmbed": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:us-east-1:000000000000:function:chunk-embed-lambda",
      "End": true,
      "Catch": [
        {
          "ErrorEquals": ["States.ALL"],
          "Next": "FailState"
        }
      ]
    },
    "FailState": {
      "Type": "Fail",
      "Cause": "Pipeline encountered an error."
    }
  }
}
```

What just happened?

This JSON tells AWS:

1. **StartAt** : Begin with the **ClassifyDocument** task.
2. **Resource** : The unique Amazon Resource Name (ARN) identifying our Lambda function.
3. **Next** : When finished, go to the **ProcessDocument** step (which points to our Extract Lambda).
4. **Catch** : If *anything* goes wrong (**States.ALL**), abort and go to **FailState** .

Here is what the state machine looks like visually:

```
graph TD
    Start((Start)) --> ClassifyDocument
    ClassifyDocument --> ProcessDocument
    ProcessDocument --> ChunkAndEmbed
    ChunkAndEmbed --> End((Success))

    ClassifyDocument -- Error --> FailState[Fail State]
    ProcessDocument -- Error --> FailState
    ChunkAndEmbed -- Error --> FailState
```

11.5 IAM and S3 Setup

Finally, we need to create the IAM roles and the S3 event trigger.

Create `infra/aws/lambda-iam-role.json`:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3:::leasegpt-uploads/*"
    },
    {
      "Effect": "Allow",
      "Action": [
        "textract:StartDocumentTextDetection",
        "textract:GetDocumentTextDetection"
      ],
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": [
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
        "logs:PutLogEvents"
      ],
      "Resource": "arn:aws:logs:*:*:*"
    }
  ]
}
```

[!NOTE]

Principle of Least Privilege: Notice how we don't give the Lambda access to the whole internet. We specifically say: You can get and put objects ONLY in the `leasegpt-uploads`

bucket. You can use Textract. You can write logs. Nothing else. If a hacker somehow takes over the Lambda, they are trapped in a very small box.

Create `infra/aws/s3-event-config.json` :

```
{
  "LambdaFunctionConfigurations": [
    {
      "Id": "TriggerStepFunctionOnUpload",
      "LambdaFunctionArn": "arn:aws:lambda:us-east-1:000000000000:function:trigger-step",
      "Events": ["s3:ObjectCreated:*"],
      "Filter": {
        "Key": {
          "FilterRules": [
            { "Name": "prefix", "Value": "uploads/" }
          ]
        }
      }
    }
  ]
}
```

Note: In reality, S3 directly triggers a tiny Lambda that just kicks off the Step Function execution.

Chapter 12: Containerization (Docker)

We've used Docker for our Lambdas, but we need to containerize our main Backend (FastAPI) and Frontend (Next.js) so they can run smoothly in the cloud.

Core Concepts Explained for Beginners

- **What is Docker?** Imagine you buy furniture from IKEA. It comes with the wood, the screws, and the Allen wrench. You don't have to go to the hardware store to find matching parts. A **Docker Container** does this for apps. It packages your app, Python/Node, and all dependencies into a single box. It runs exactly the same on a Windows laptop, a Mac, or a server in Iceland.
- **Image vs Container:** A Docker **Image** is the instruction manual (the recipe). A Docker **Container** is the running application (the cooked meal). You can spin up ten identical containers from one image.
- **Multi-stage Build:** If you bake a cake, your kitchen counter gets covered in flour and eggshells. You don't serve the cake on the messy counter; you move it to a clean plate. A **multi-stage build** does this: it builds your app in a messy "Stage 1" container, then copies

only the finished, clean app into a tiny "Stage 2" container. This keeps your final image small and fast!

12.1 Backend Dockerfile

Create `backend/Dockerfile` :

```
# backend/Dockerfile

# --- STAGE 1: Build & Install Dependencies (The messy kitchen) ---
FROM python:3.11-slim AS builder

# Set the working directory inside the container
WORKDIR /app

# Install build tools required for some Python packages (like psycpg2)
RUN apt-get update && apt-get install -y gcc libpq-dev

# Copy only requirements first (this helps Docker cache dependencies!)
COPY requirements.txt .

# Install dependencies into a specific folder so we can copy them later
RUN pip install --user -r requirements.txt

# --- STAGE 2: Final Image (The clean plate) ---
FROM python:3.11-slim

WORKDIR /app

# We still need the runtime libraries for postgres
RUN apt-get update && apt-get install -y libpq5 curl && rm -rf /var/lib/apt/lists/*

# Copy the installed dependencies from the 'builder' stage
COPY --from=builder /root/.local /root/.local

# Ensure the installed scripts are on the PATH
ENV PATH=/root/.local/bin:$PATH

# Copy our actual FastAPI application code
COPY . .

# Expose the port FastAPI runs on
EXPOSE 8000

# Add a health check so Docker knows if our app crashes
HEALTHCHECK --interval=30s --timeout=30s --start-period=5s --retries=3 \
  CMD curl -f http://localhost:8000/health || exit 1

# The command to start the web server
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

12.2 Frontend Dockerfile

Create `frontend/Dockerfile` :

```

# frontend/Dockerfile

# --- STAGE 1: Install & Build ---
FROM node:18-alpine AS builder

WORKDIR /app

# Copy package.json and install dependencies
COPY package.json package-lock.json ./
RUN npm ci

# Copy the rest of the Next.js code and build it
COPY . .
# We use Next.js 'standalone' output mode.
# It creates a minimal server with ONLY the files needed to run.
RUN npm run build

# --- STAGE 2: Run ---
FROM node:18-alpine AS runner

WORKDIR /app

# Set environment to production
ENV NODE_ENV production

# Copy only the necessary files from the builder stage
COPY --from=builder /app/public ./public
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./next/static

EXPOSE 3000

CMD ["node", "server.js"]

```

12.3 The Master Docker Compose File

Now we connect EVERYTHING. Update `infra/docker/docker-compose.yml` :

```
# infra/docker/docker-compose.yml
version: '3.8'

services:
  # 1. Our PostgreSQL Database with pgvector
  postgres:
    image: ankane/pgvector:v0.5.1
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: ${DB_PASSWORD:-postgres}
      POSTGRES_DB: leasegpt
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 5s
      retries: 5

  # 2. Redis (Used for LangGraph memory and agent state)
  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"

  # 3. Ollama (Local AI models for embeddings and chat)
  ollama:
    image: ollama/ollama:latest
    ports:
      - "11434:11434"
    volumes:
      - ollama_data:/root/.ollama

  # 4. LocalStack (Fakes AWS S3, Lambda, Step Functions locally)
  localstack:
    image: localstack/localstack
    ports:
      - "4566:4566"
    environment:
      - SERVICES=s3,lambda,stepfunctions,iam
      - DEBUG=1
    volumes:
      - "/var/run/docker.sock:/var/run/docker.sock"

  # 5. FastAPI Backend
  backend:
    build:
      context: ../../backend
    ports:
      - "8000:8000"
    environment:
      - DB_HOST=postgres
      - REDIS_HOST=redis
      - OLLAMA_HOST=http://ollama:11434
      - AWS_ENDPOINT_URL=http://localstack:4566
```

```

    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_started

# 6. Next.js Frontend
frontend:
  build:
    context: ../../frontend
  ports:
    - "3000:3000"
  environment:
    - NEXT_PUBLIC_API_URL=http://localhost:8000
  depends_on:
    - backend

volumes:
  pgdata:
  ollama_data:

```

What just happened?

This file is the conductor of our orchestra. With one command (`docker-compose up --build`), Docker will:

1. Download Postgres, Redis, Ollama, and LocalStack.
2. Build our Backend Python container.
3. Build our Frontend Next.js container.
4. Network them all together so they can talk to each other (e.g., the backend can reach the database at `http://postgres:5432`).

Chapter 13: Kubernetes Deployment

Docker is great for running things on one laptop. But what if LeaseGPT gets featured on the news and 10,000 people upload PDFs at once? One laptop will melt.

Core Concepts Explained for Beginners

- **What is Kubernetes (K8s)?** Imagine Docker is a single food truck. **Kubernetes** is the fleet manager for 500 food trucks. You tell Kubernetes, "I always want exactly 3 backend trucks running." If a truck's engine explodes (a server crashes), Kubernetes instantly deploys a new truck to replace it. It manages dozens of servers and automatically spreads your Docker containers across them.
- **Pod:** The smallest unit in Kubernetes. Usually, it's just a thin wrapper around a single Docker container.

- **Deployment:** The blueprint. You write a Deployment YAML saying, "Run 2 copies (replicas) of my backend Pod."
- **Service:** The phonebook. Because Pods are constantly dying and being reborn with new IP addresses, a Service gives a stable, permanent address (like `http://backend-service`) that routes traffic to whichever Pods are currently alive.
- **Ingress:** The receptionist. It sits at the front door of your cluster, takes traffic from the public internet (e.g., `leasegpt.com`), and routes it to the correct Service.
- **StatefulSet:** Deployments are great for web servers because web servers are replaceable. Databases (like Postgres) are NOT replaceable—they hold data! A StatefulSet gives a pod a sticky identity and a permanent hard drive (Persistent Volume) so it never loses data.
- **kind (Kubernetes in Docker):** A tool that creates a full, working Kubernetes cluster *inside* your local Docker. It's for practicing K8s without paying for expensive cloud servers.

Let's write our Kubernetes files using **Kustomize**, which allows us to define a "Base" configuration and tweak it for "Dev" or "Production".

13.1 Base Configuration

Create `infra/k8s/base/namespace.yml` :

```
# Creates an isolated sandbox in the cluster for our app
apiVersion: v1
kind: Namespace
metadata:
  name: leasegpt
```

Create `infra/k8s/base/postgres-statefulset.yml` :


```

apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
  namespace: leasegpt
spec:
  serviceName: "postgres"
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: ankane/pgvector:v0.5.1
          env:
            - name: POSTGRES_USER
              value: postgres
            - name: POSTGRES_PASSWORD
              value: postgres # In production, this would use a K8s Secret!
            - name: POSTGRES_DB
              value: leasegpt
          ports:
            - containerPort: 5432
          volumeMounts:
            - name: pgdata
              mountPath: /var/lib/postgresql/data
      # This asks Kubernetes to provision a permanent 10GB hard drive
      volumeClaimTemplates:
        - metadata:
            name: pgdata
          spec:
            accessModes: [ "ReadWriteOnce" ]
            resources:
              requests:
                storage: 10Gi

---
# The Service to route traffic to Postgres
apiVersion: v1
kind: Service
metadata:
  name: postgres-service
  namespace: leasegpt
spec:
  selector:
    app: postgres
  ports:
    - protocol: TCP
      port: 5432
      targetPort: 5432

```

Create `infra/k8s/base/backend-deployment.yml` :

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: leasegpt
spec:
  replicas: 2 # Run TWO copies of our backend for high availability!
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: leasegpt-backend:latest
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8000
          envFrom:
            - configMapRef:
                name: app-config
          # Kubernetes uses this to check if the app is frozen
          livenessProbe:
            httpGet:
              path: /health
              port: 8000
            initialDelaySeconds: 10
            periodSeconds: 15

---
apiVersion: v1
kind: Service
metadata:
  name: backend-service
  namespace: leasegpt
spec:
  selector:
    app: backend
  ports:
    - protocol: TCP
      port: 8000
      targetPort: 8000

```

Create `infra/k8s/base/frontend-deployment.yml` :

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
  namespace: leasegpt
spec:
  replicas: 2
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
      - name: frontend
        image: leasegpt-frontend:latest
        imagePullPolicy: IfNotPresent
        ports:
        - containerPort: 3000
        envFrom:
        - configMapRef:
            name: app-config

---
apiVersion: v1
kind: Service
metadata:
  name: frontend-service
  namespace: leasegpt
spec:
  selector:
    app: frontend
  ports:
  - protocol: TCP
    port: 3000
    targetPort: 3000

```

Create `infra/k8s/base/ingress.yml` :

```
# The Receptionist: Routes web traffic based on the URL path
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: leasegpt-ingress
  namespace: leasegpt
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
    - http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: backend-service
                port:
                  number: 8000
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend-service
                port:
                  number: 3000
```

Create `infra/k8s/base/configmap.yml`:

```
# Stores non-secret environment variables
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  namespace: leasegpt
data:
  DB_HOST: "postgres-service"
  REDIS_HOST: "redis-service"
  NEXT_PUBLIC_API_URL: "/api"
```

Create `infra/k8s/base/kustomization.yml`:

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - namespace.yml
  - configmap.yml
  - postgres-statefulset.yml
  - backend-deployment.yml
  - frontend-deployment.yml
  - ingress.yml
```

13.2 Environment Overrides

Create `infra/k8s/overlays/dev/kustomization.yml` :

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
- ../../base
# In dev, we only want 1 replica to save laptop battery
patches:
- target:
    kind: Deployment
    name: backend
  patch: |-
    - op: replace
      path: /spec/replicas
      value: 1
```

Chapter 14: CI/CD (GitHub Actions)

We don't want to manually run tests and deploy to servers. We want a robot to do it every time we push code to GitHub.

Core Concepts Explained for Beginners

- **What is CI/CD?** Continuous Integration / Continuous Deployment. Imagine having a fastidious assistant. Every time you write an essay, you hand it to them. They check the spelling (Linting), read the logic (Testing), and if it's perfect, they automatically mail it to the publisher (Deployment).
- **GitHub Actions:** GitHub's built-in robot assistant.
- **Workflow:** A YAML file telling the robot exactly what tasks to run (e.g., "Run tests when someone opens a Pull Request").

14.1 Continuous Integration (CI)

This runs on every Pull Request to ensure nobody merges broken code.

Create `.github/workflows/ci.yml` :

```

name: CI Pipeline

# Trigger this workflow when a Pull Request is opened against 'main'
on:
  pull_request:
    branches: [ main ]

jobs:
  # Job 1: Check Python Code Quality
  backend-lint-test:
    runs-on: ubuntu-latest

    # We need a temporary database to run integration tests
    services:
      postgres:
        image: ankane/pgvector:v0.5.1
        env:
          POSTGRES_USER: postgres
          POSTGRES_PASSWORD: postgres
          POSTGRES_DB: leasegpt_test
        ports:
          - 5432:5432

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'
          cache: 'pip' # Speeds up subsequent runs

      - name: Install dependencies
        run: |
          cd backend
          pip install -r requirements.txt
          pip install ruff mypy pytest pytest-asyncio

      - name: Lint with Ruff (checks for formatting and style errors)
        run: cd backend && ruff check .

      - name: Type check with mypy
        run: cd backend && mypy .

      - name: Run Unit Tests
        env:
          DB_HOST: localhost
          DB_NAME: leasegpt_test
        run: cd backend && pytest tests/unit/ -v

  # Job 2: Check Frontend Code Quality
  frontend-lint-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

```

```
- name: Set up Node.js
  uses: actions/setup-node@v4
  with:
    node-version: '18'
    cache: 'npm'

- name: Install dependencies
  run: cd frontend && npm ci

- name: Lint (ESLint)
  run: cd frontend && npm run lint

- name: Type check (TypeScript)
  run: cd frontend && npm run type-check

- name: Run Tests (Jest)
  run: cd frontend && npm run test
```

14.2 Continuous Deployment (CD) to Production

When we tag a release (e.g., `v1.0.0`), we build Docker images and update Kubernetes.

Create `.github/workflows/cd-production.yml`:

```

name: Deploy to Production

on:
  push:
    tags:
      - 'v*' # Triggered when we push a tag like v1.0.0

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      # 1. Login to a Container Registry (where Docker images are hosted)
      - name: Login to GitHub Container Registry
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${github.actor}
          password: ${secrets.GITHUB_TOKEN}

      # 2. Build and push the backend image
      - name: Build and push Backend
        uses: docker/build-push-action@v5
        with:
          context: ./backend
          push: true
          tags: ghcr.io/${github.repository}/backend:${github.ref_name}

      # 3. Connect to our Kubernetes Cluster
      # (Assuming AWS EKS in this example, credentials stored in GitHub Secrets)
      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${secrets.AWS_ACCESS_KEY_ID}
          aws-secret-access-key: ${secrets.AWS_SECRET_ACCESS_KEY}
          aws-region: us-east-1

      - name: Update KubeConfig
        run: aws eks update-kubeconfig --name leasegpt-cluster --region us-east-1

      # 4. Tell Kubernetes to use the new Docker images (Rolling Update)
      - name: Deploy to K8s
        run: |
          cd infra/k8s/overlays/production

          # Update the kustomization file with the new image tags
          kustomize edit set image leasegpt-backend=ghcr.io/${github.repository}/backend:${github.ref_name}
          kustomize edit set image leasegpt-frontend=ghcr.io/${github.repository}/frontend:${github.ref_name}

          # Apply the changes to the cluster
          kustomize build . | kubectl apply -f -

      # 5. Wait for the new pods to become healthy
      - name: Verify deployment
        run: |

```



```
kubectl rollout status deployment/backend -n leasegpt
kubectl rollout status deployment/frontend -n leasegpt
```

Chapter 15: Testing & Evaluation

AI applications are notoriously hard to test. Standard unit tests ensure our API endpoints work, but how do we know if the AI is hallucinating answers about the lease? We need **Evaluation**.

Core Concepts Explained for Beginners

- **Unit Testing (pytest):** Checking individual ingredients. Does this one Python function that splits text work properly?
- **Golden Dataset:** A set of 10-20 manually curated Questions and *Perfect* Answers. We know these are 100% correct.
- **Ragas (RAG Assessment):** A framework that takes our Golden Dataset, runs the questions through our LangGraph agent, and uses an LLM as a "judge" to score the AI's answers on things like:
 - **Faithfulness:** Did the AI make stuff up?
 - **Context Precision:** Did the database retrieve the right paragraphs from the lease?

15.1 Unit Tests Setup

Create `backend/tests/conftest.py` (Pytest Fixtures):

```
# backend/tests/conftest.py
import pytest
from fastapi.testclient import TestClient
from main import app

# A Fixture is like a setup tool.
# Pytest runs this before a test, hands the 'client' to the test, and cleans up after.
@pytest.fixture
def client():
    # TestClient allows us to make fake HTTP requests to our API without starting a real server
    with TestClient(app) as test_client:
        yield test_client
```

Create `backend/tests/unit/test_schemas.py` :

```
# backend/tests/unit/test_schemas.py
from schemas.chat import ChatRequest

def test_chat_request_validation():
    # If we pass a valid message, it shouldn't crash
    req = ChatRequest(lease_id="123", message="Who is the tenant?")
    assert req.message == "Who is the tenant?"

    # If we pass empty strings, Pydantic should catch it (assuming we added validators)
    import pytest
    from pydantic import ValidationError
    with pytest.raises(ValidationError):
        ChatRequest(lease_id="", message="")
```

15.2 AI Evaluation with Ragas

Create `backend/tests/evaluation/golden_qa.json` :

```
[
  {
    "question": "What is the monthly rent?",
    "ground_truth": "The monthly rent is $2,500.",
    "context": "The tenant agrees to pay a monthly rent of $2,500 due on the 1st of each month.",
  },
  {
    "question": "Are pets allowed?",
    "ground_truth": "No, pets are strictly prohibited.",
    "context": "Under no circumstances are animals or pets of any kind permitted on the premises.",
  }
]
```

Create `backend/tests/evaluation/test_ragas.py` :

```

# backend/tests/evaluation/test_ragas.py
import json
import pytest
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy
from datasets import Dataset

# Imagine we imported our LangGraph agent here
# from agent.graph import process_chat_message

@pytest.mark.asyncio
async def test_rag_pipeline_quality():
    """
    This test doesn't just check if code runs; it checks if the AI is SMART.
    """
    # 1. Load the golden dataset
    with open("tests/evaluation/golden_qa.json", "r") as f:
        golden_data = json.load(f)

    questions = []
    ground_truths = []
    answers = []
    contexts = []

    for item in golden_data:
        questions.append(item["question"])
        ground_truths.append(item["ground_truth"])

        # 2. ASK OUR AI THE QUESTION!
        # (Mocking the AI response for this example)
        # response = await process_chat_message(item["question"], "fake_lease_id")
        # answers.append(response.answer)
        # contexts.append(response.retrieved_chunks)

        # Fake successful AI response for demonstration
        answers.append(item["ground_truth"])
        contexts.append([item["context"]])

    # 3. Package the data for Ragas
    data = {
        "question": questions,
        "answer": answers,
        "contexts": contexts,
        "ground_truth": ground_truths
    }
    dataset = Dataset.from_dict(data)

    # 4. Have an LLM "Judge" evaluate the results!
    # Faithfulness checks if the answer is directly supported by the retrieved contexts
    # Answer Relevancy checks if the answer actually addresses the question.
    result = evaluate(
        dataset,
        metrics=[faithfulness, answer_relevancy]
    )

    print(f"Ragas Evaluation Scores: {result}")

```

```
# 5. Assert our AI meets a minimum quality bar (e.g., 80% accurate)
# If the score drops below 0.8, the CI/CD pipeline will fail, preventing a bad AI f
assert result['faithfulness'] > 0.80
assert result['answer_relevancy'] > 0.80
```

Chapter 16: Running the Complete System

You've built an incredible, production-ready AI application. Here is how you spin up the entire universe on your Windows machine.

Step 1: Start Docker Compose

Open your terminal (PowerShell or WSL2) and navigate to the docker folder:

```
cd infra/docker
docker-compose up --build -d
```

The `-d` flag runs it in the background.

Step 2: Initialize LocalStack (AWS Mock)

We need to create our fake S3 bucket locally.

```
# This uses the AWS CLI to talk to our LocalStack container instead of the real AWS
aws --endpoint-url=http://localhost:4566 s3 mb s3://leasegpt-uploads
```

Step 3: Run Database Migrations

Apply our pgvector schema:

```
cd ../../backend
# Wait for the postgres container to be healthy, then:
# (Assuming you use Alembic for migrations, or simply run a python script to create tables)
python init_db.py
```

Step 4: Access the UI

Open your browser and navigate to:

`http://localhost:3000`

1. Upload a PDF.

2. Watch the logs (`docker-compose logs -f`) to see the file hit the FastAPI backend, get saved to LocalStack S3, and trigger the Lambdas.
3. Once processed, type "What is the late fee policy?" in the chat window.
4. Watch LangGraph route the query, retrieve vectors from PostgreSQL, and stream the answer back to the UI!

Troubleshooting

- **Error: psycopg2 fails to install in Docker:** Ensure `postgresql-devel` and `gcc` are in your Dockerfile `RUN apt-get` or `yum install` commands.
- **Next.js says "Connection Refused":** Make sure the backend is fully running. Check `docker-compose ps`.
- **Ollama is slow:** Running LLMs locally requires serious RAM and CPU. If it's too slow, edit `docker-compose.yml` to remove the Ollama container and replace the `OLLAMA_HOST` with a real OpenAI API key in the backend.

Conclusion

Congratulations! 🎉 Over the course of 5 parts, you have progressed from zero to a full-stack AI engineer. You built a Retrieval-Augmented Generation (RAG) system, orchestrated agentic workflows with LangGraph, deployed serverless infrastructure on AWS, containerized with Docker, prepared for scaling with Kubernetes, and automated quality assurance with CI/CD and Ragas.

You didn't just build a toy script; you built **LeaseGPT** using real-world, enterprise-grade architecture.

Take a breath, grab a coffee, and celebrate. You've earned it!
